

Universidad de La Habana
Facultad de Matemática y Computación



**El vacío de la intersección de una
gramática de concatenación de rango con
un lenguaje regular finito como marco
para el SAT**

Autor:

Luis Alejandro Arteaga Morales

Tutores:

MSc. Fernando Raul Rodriguez Flores

Lic. Raudel Alejandro Gómez Molina

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

Junio de 2026

A mi abuelo

Agradecimientos

A mi madre, por haber dedicado cada pedacito de su existencia a mi hermano y a mí, por tantos desvelos y por brindarme siempre su amor y su apoyo.

A mi padre, por haberme enseñado a ser la persona que soy hoy, por inculcarme sus valores y por enseñarme a nunca rendirme, a no quedarme en mi comodidad e ir por lo que quiero, y a confiar en que las cosas saldrán bien.

A mi hermano, por haber sido siempre la luz que guió mi camino en todo momento; de no ser por ti, no estaría donde estoy hoy.

Al resto de mi familia, por haberme apoyado incondicionalmente en este proceso lleno de desafíos y haber sabido comprender cuando no podía estar; por darme la posibilidad de enfocarme en hacer lo que quería y por confiar en mí. En especial, a mis abuelas y a mis primas, por haberme tenido que convertir en su ejemplo; eso también me hizo mejor.

A mi abuelo, que no pudo estar para ver este momento: gracias por inculcarme el hábito de leer desde pequeño y la curiosidad por comprender el mundo.

A mis amigos Pablo, Diego, Francisco, Mauricio y Jossué, por todos los momentos, los cafés y las noches de estudio interminables que compartimos en la bequita; ustedes hicieron que este viaje fuera más llevadero y más divertido.

A Darío, por haber formado parte del grupo también y por el conocimiento compartido.

A mis compañeros de ICPC, Alvarito, Manzano, Ahmed, por todos los rounds luchando por un ACCEPTED.

A todos los del claustro de EDA, Cartaya, Alberto, Alex, Raudel, Eduardo y los chicos nuevos, por todos los momentos que compartimos y por aportar nuestro granito de arena para enseñar a las personas.

A mis tutores, Fernando y Raudel, por su infinita paciencia y por confiar en mí hasta el último momento, por todo el conocimiento y la sabiduría compartidos, y por enseñarme un poco más sobre cómo se hace ciencia.

A Arlin, por haber sido parte de mi familia y mi mejor amiga durante estos duros años de carrera. Gracias por el apoyo y las noches de conversaciones sin sentido.

A Mary, gracias por haber sido parte de este proceso, por el apoyo brindado desde que nos conocemos y por también haber sido familia.

Al resto de mis amigos de Matanzas, Carlos, Pedro, Sthefano: gracias por haberme sabido comprender en cada momento y por las charlas de apoyo emocional.

Opinión del tutor

En este trabajo se aborda el problema de la satisfacibilidad booleana (SAT) desde una perspectiva original dentro de la teoría de lenguajes formales, en la línea de investigación que reduce la satisfacibilidad de una fórmula al vacío de la intersección entre un lenguaje regular de interpretaciones y una gramática que impone la consistencia entre las ocurrencias de cada variable booleana. La principal contribución de esta tesis es el estudio sistemático de distintas clases gramaticales para modelar dicha consistencia: desde las gramáticas libres de contexto, pasando por las RCG simples, hasta llegar a las RCG generales con no linealidad. Se muestra que una RCG puede reconocer la consistencia de cualquier fórmula de SAT con aridad uno, pero precisamente la no linealidad que lo hace posible elimina la garantía de que el vacío sea decidible en tiempo polinomial. Se construye además la gramática producto, un objeto unificado que permite abordar el problema del vacío, y se caracterizan su tamaño y el costo de construirlo, dejando abiertas preguntas fundamentales sobre su potencial para futuras investigaciones.

Para realizar este trabajo, Luis Alejandro tuvo que incursionar en temas que no forman parte de su plan de estudios, como las gramáticas de concatenación de rango (RCG), así como en técnicas avanzadas de lenguajes formales. Su capacidad para comprender formalismos complejos y para navegar con soltura entre la teoría de lenguajes, la complejidad computacional y la lógica proposicional es digna de elogio.

Queremos resaltar la independencia absoluta con la que Luis Alejandro condujo su investigación, así como su rigor y el pensamiento crítico. Consideramos que estamos en presencia de un trabajo excelente, realizado por un excelente científico de la computación.



MSc. Fernando Raul Rodriguez Flores
Facultad de Matemática y
Computación
Universidad de La Habana.



Lic. Raudel Alejandro Gómez Molina
Facultad de Matemática y
Computación
Universidad de La Habana.

Resumen

El problema de la satisfacibilidad booleana (SAT) se estudia en este trabajo desde la teoría de lenguajes formales. En la línea de investigación en la que se inscribe, la satisfacibilidad de una fórmula se reduce al vacío de la intersección de un lenguaje regular de interpretaciones con una gramática que impone la consistencia entre las ocurrencias de cada variable booleana. Con gramáticas menos expresivas esa reducción se limita a fórmulas con cierto orden de las ocurrencias: una gramática libre del contexto no admite cruces, y en una de concatenación de rango simple (sRCG) la aridad crece con la fórmula. En un intento de resolver más fórmulas en tiempo polinomial se ataca el problema con gramáticas de concatenación de rango (RCG): con su no linealidad se modela la consistencia de más fórmulas y se mantiene mínima la aridad, como se muestra con una RCG que reconoce la consistencia de cualquier fórmula de SAT con aridad uno; pero por esa misma no linealidad se pierde la garantía directa de que el vacío sea polinomial. Se da un algoritmo de fuerza bruta que decide ese vacío: se enumeran las cadenas del autómata y se reconocen con la RCG; su costo es exponencial, así que no es una vía factible, pero muestra por qué conviene estudiar el problema sobre un único objeto. Se construye ese objeto, la gramática producto, y se caracterizan su tamaño y el costo de construirlo; sobre él se puede abordar el vacío: hallar un método polinomial para todas las fórmulas, lo que implicaría $P = NP$; probar que no existe; o hallarlo al restringir a una subclase. Aunque su tamaño no es polinomial, queda como objeto de estudio: eliminar sus cláusulas no productivas y analizar el costo de decidir el vacío se proponen como trabajo futuro.

Palabras clave: satisfacibilidad booleana, gramáticas de concatenación de rango, vacío de la intersección, lenguaje regular finito, no linealidad.

Abstract

The Boolean satisfiability problem (SAT) is studied in this work from formal language theory. Within the line of research to which it belongs, the satisfiability of a formula reduces to the emptiness of the intersection of a regular language of interpretations with a grammar that enforces consistency among the occurrences of each Boolean variable. With less expressive grammars this reduction is limited to formulas with a certain order of occurrences: a context-free grammar admits no crossings, and with a simple range concatenation grammar (sRCG) the arity grows with the formula. In an attempt to solve more formulas in polynomial time, the problem is approached with range concatenation grammars (RCG): with their non-linearity the consistency of more formulas is modeled while the arity is kept minimal, as shown by an RCG that recognizes the consistency of any SAT formula with arity one; but with that same non-linearity the direct guarantee that the emptiness is polynomial is lost. A brute-force algorithm that decides this emptiness is given: the strings of the automaton are enumerated and recognized with the RCG; its cost is exponential, so enumeration is not a feasible route, but it shows why the problem is better studied on a single object. This object, the product grammar, is constructed, and its size and the cost of constructing it are characterized; on it the emptiness can be addressed: finding a polynomial method for all formulas, which would imply $P = NP$; proving that none exists; or finding one only by restricting the modeling to a subclass. Although its size is not polynomial, it remains an object of study: removing its non-productive clauses and analyzing the cost of deciding the emptiness are proposed as future work.

Keywords: boolean satisfiability, range concatenation grammars, emptiness of the intersection, finite regular language, non-linearity.

Índice general

Introducción	1
1. Preliminares	4
1.1. Teoría de Lenguajes	4
1.1.1. Conceptos básicos	4
1.1.2. Gramáticas	5
1.1.3. Jerarquía de Chomsky	6
1.2. Autómatas finitos	8
1.3. Complejidad computacional	9
1.3.1. Notación asintótica	9
1.3.2. Reducción polinomial	9
1.3.3. Clases de problemas	10
1.4. Problema de la satisfacibilidad booleana	10
1.4.1. Elementos del SAT	11
1.4.2. SAT como problema <i>NP</i> -Completo y variantes polinomiales	12
2. Gramáticas de concatenación de rango y el SAT como lenguaje formal	13
2.1. Gramáticas de concatenación de rango	13
2.1.1. Presentación intuitiva	14
2.1.2. Definiciones formales	16
2.1.3. Ejemplo de reconocimiento	18
2.2. Propiedades de las RCG	19
2.2.1. Complejidad y caracterización en términos de <i>P</i>	20
2.3. Antecedentes en el estudio del SAT mediante formalismos de la teoría de lenguajes	20
2.3.1. El autómata de las interpretaciones	21
2.3.2. La consistencia entre ocurrencias y la clase de fórmulas	24

3. Vacío de la intersección de una RCG con un lenguaje regular finito	27
3.1. Lenguajes regulares finitos	28
3.2. Autómatas acíclicos	28
3.3. Problema de vacío de la intersección	30
3.4. Algoritmo de decisión	30
3.4.1. Correctitud	32
3.4.2. Complejidad	33
4. El SAT como vacío de la intersección de una RCG con un lenguaje regular finito	36
4.1. Modelación del SAT con RCG	36
4.2. Consecuencias para el SAT y subclases tratables	41
5. Construcción de la gramática producto de una gramática de concatenación de rango y un lenguaje regular finito	44
5.1. El rango sobre un autómata	44
5.2. El rango de un argumento sobre un autómata	46
5.3. Construcción de la gramática producto	47
5.3.1. Algoritmo	48
5.3.2. Restricción sobre la gramática de entrada	51
5.3.3. Variables repetidas en una cláusula	51
5.3.4. Pseudocódigo	52
5.3.5. Correctitud	54
5.3.6. Tamaño y complejidad	57
Conclusiones y recomendaciones	59
Referencias	61

Índice de figuras

1.1. Jerarquía de Chomsky.	8
2.1. Autómata booleano de una ocurrencia de variable.	21
2.2. Autómata booleano de la negación	22
2.3. Autómata booleano de la disyunción	22
2.4. Autómata booleano de la conjunción	23
2.5. Autómatas de las subfórmulas de A	23
2.6. Autómata booleano de $A = x_1 \wedge (x_2 \vee x_1)$	24
2.7. Órdenes de ocurrencias: anidado y con cruce	25
4.1. Partición del argumento de Eq_v	37
4.2. Subcadenas cubiertas por cada W_k	39
4.3. Árbol de derivación de 101 en G_{cons}	40
4.4. La cadena 110 no es consistente	41
5.1. Caminos de los ejemplos de $\hat{\delta}$	46
5.2. Árbol de derivación de 101 en G'	50

Introducción

El problema de la satisfacibilidad booleana (SAT) ocupa un lugar central en las ciencias de la computación. Fue el primer problema que se demostró NP-completo en 1971, un resultado de Stephen Cook [8]. Por tanto, todo problema de la clase NP se reduce a él, y decidirlo en tiempo polinomial resolvería si $P = NP$. A la vez, el SAT tiene aplicaciones en la criptografía, el diseño y la verificación de circuitos, los problemas de satisfacción de restricciones (CSP) y la programación lógica [12]. Por eso interesa entender qué lo hace difícil y delimitar los casos que se resuelven en tiempo polinomial, como 2-SAT, Horn-SAT y XOR-SAT [17].

En una línea de investigación de la Facultad de Matemática y Computación de la Universidad de La Habana, el SAT se aborda desde la teoría de lenguajes formales. A cada fórmula booleana se le asocian un lenguaje regular y una gramática. Las cadenas del lenguaje regular representan las interpretaciones de sus variables que satisfacen la fórmula, asumiendo que no hay variables repetidas, y la gramática impone la consistencia: que las ocurrencias de una misma variable tomen el mismo valor. La satisfacibilidad de la fórmula equivale entonces a que la intersección de ambos lenguajes no sea vacía. Así, el SAT se traduce en un problema sobre lenguajes formales, y la dificultad de decidirlo se traslada al vacío de esa intersección.

Esta línea cuenta con antecedentes según el formalismo de la gramática de consistencia. En la satisfacibilidad booleana libre del contexto (SATCF), la consistencia se impone con una gramática libre del contexto, y el SAT se decide en tiempo polinomial para las fórmulas con orden de ocurrencias libre del contexto [10]. En la satisfacibilidad booleana de concatenación de rango simple (SATSRC), la consistencia se impone con una gramática de concatenación de rango simple, que admite cruces entre ocurrencias, y el SAT se decide en tiempo polinomial mientras la aridad necesaria esté acotada [1]. Las gramáticas de concatenación de rango (RCG) son el siguiente formalismo de esta línea.

En una RCG, los no terminales, llamados predicados, reconocen tuplas de subcadenas en lugar de una sola, y la aridad de un predicado es la cantidad de subcadenas de la tupla. Una RCG es no lineal cuando una misma variable aparece más de una vez en el lado derecho de una regla, y así se puede exigir que distintas subcadenas coincidan.

Con las RCG lineales se reconocen los mismos lenguajes que con las sRCG, de modo que con ellas se modela la consistencia de las mismas fórmulas. Para modelar la consistencia de más fórmulas, y resolver así más instancias del SAT, hace falta la no linealidad de las RCG [6].

Al pasar de las gramáticas libres del contexto y las de concatenación de rango simple a las RCG no se conserva la tratabilidad del vacío de la intersección. Bertsch y Nederhof demostraron que el vacío de la intersección de una RCG con un lenguaje regular arbitrario es indecidible, y que, cuando el lenguaje regular es finito, decidir si la intersección es no vacía es NP-completo [4]. El lenguaje regular de las interpretaciones de una fórmula es finito, así que este enfoque queda en el caso decidable.

Con la no linealidad de las RCG se expresa la consistencia de cualquier fórmula, sin la restricción de orden de SATCF y SATSRC, pero el problema queda fuera de las subclases con vacío de la intersección polinomial conocidas. Aun así, ese vacío se decide al enumerar las cadenas del autómata y reconocer cada una con la gramática de consistencia, pero en tiempo exponencial y con los dos formalismos por separado. Falta, entonces, una representación de esa intersección sobre la que estudiar su vacío en un solo formalismo.

Teniendo en cuenta lo anterior, surge la siguiente pregunta: ¿es posible expresar la intersección de una gramática de concatenación de rango con un lenguaje regular finito mediante una sola gramática de concatenación de rango, sobre la que estudiar el vacío que decide el SAT?

El objeto de estudio de este trabajo es la resolución del SAT mediante formalismos de la teoría de lenguajes; dentro de ella, el campo de acción es el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito.

El objetivo general de este trabajo es estudiar el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito como marco para la solución del SAT, y construir una representación de esa intersección sobre la que abordar dicho problema.

Para alcanzar ese objetivo general se plantean los siguientes objetivos específicos:

- Estudiar las gramáticas de concatenación de rango y los antecedentes del SAT abordado mediante formalismos de la teoría de lenguajes.
- Diseñar un algoritmo que decida el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito y caracterizar su costo.
- Modelar el SAT como el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito, asociando a cada fórmula un autómata booleano y una gramática que impone la consistencia entre las ocurrencias de cada variable.

- Construir la gramática producto que reconoce la intersección de una gramática de concatenación de rango con un lenguaje regular finito, demostrar su correctitud y caracterizar su tamaño y el costo de construirla.

La propuesta de solución parte de un algoritmo de fuerza bruta que decide el vacío de la intersección de una RCG con un lenguaje regular finito, y se caracteriza su costo. La consistencia se impone con una RCG de aridad uno, la mínima posible, gracias a la no linealidad, y se construye la gramática producto, cuyo lenguaje es exactamente esa intersección.

Como resultado, se obtiene un marco uniforme para el SAT en torno al vacío de la intersección de una RCG con un lenguaje regular finito. El algoritmo de decisión tiene costo exponencial en el peor caso. La gramática de consistencia tiene aridad uno para cualquier fórmula; a cambio, se pierde la garantía directa de que el vacío sea polinomial. Se construye la gramática producto y se demuestra su correctitud; su tamaño puede ser exponencial, con muchos predicados y cláusulas no productivos. Queda como problema abierto eliminarlos y analizar el costo de decidir el vacío sobre la gramática resultante.

Por último, es importante mencionar el uso de la inteligencia artificial en la elaboración de este trabajo. Como apoyo en la revisión bibliográfica, se usó para buscar y organizar información en bases de datos académicas, y así se accedió a bibliografía actualizada y pertinente para el marco teórico. La opinión de los tutores presentada en este documento fue generada (por uno de los tutores, con el consentimiento del otro) usando modelos grandes de lenguaje, en particular DeepSeek. Para ello se usó el resumen de la tesis y las características personales del estudiante que resultaron relevantes para el desarrollo del trabajo. Para alcanzar el tono de escritura propio de los tutores, se proporcionaron como ejemplos los mismos elementos (resumen de la tesis y características del estudiante) de opiniones del tutor de cursos anteriores.

La estructura de este documento es la siguiente. En el capítulo 1 se presentan los preliminares de la teoría de lenguajes, los autómatas finitos, la complejidad computacional y el SAT. En el capítulo 2 se presentan las gramáticas de concatenación de rango y los antecedentes del SAT mediante formalismos de la teoría de lenguajes. En el capítulo 3 se define el problema del vacío de la intersección de una RCG con un lenguaje regular finito y se da un algoritmo de decisión. En el capítulo 4 se modela el SAT como ese vacío, con una RCG de consistencia para cualquier fórmula. En el capítulo 5 se construye la gramática producto y se caracterizan su tamaño y el costo de construirla. Por último, en las conclusiones y recomendaciones se recogen los resultados y las líneas de investigación que se proponen.

Capítulo 1

Preliminares

En este capítulo se presentan las definiciones y conceptos que serán usados en el resto del trabajo. En la sección 1.1 se introducen las nociones básicas de la teoría de lenguajes. Después, en la sección 1.2, se presentan los autómatas finitos. Por último, en las secciones 1.3 y 1.4, se introduce la complejidad computacional y se presenta el problema de la satisfacibilidad booleana con sus variantes polinomiales.

1.1. Teoría de Lenguajes

En esta sección se presentan los conceptos de la teoría de lenguajes que sirven de base al contenido de los capítulos posteriores. La sección se organiza en tres partes. Primero se introducen los conceptos de alfabeto, cadena y lenguaje, junto con los problemas de la palabra y del vacío. Después se definen las gramáticas, un mecanismo que permite describir lenguajes formales. Para finalizar se presenta la jerarquía de Chomsky, que clasifica los lenguajes formales según las gramáticas que los generan.

1.1.1. Conceptos básicos

Los conceptos básicos de la teoría de lenguajes formales son: alfabeto, cadena y lenguaje. Un **alfabeto**, denotado como Σ , es un conjunto finito y no vacío de símbolos. Una **cadena** es una sucesión finita de símbolos del alfabeto. Un **lenguaje** es un conjunto de cadenas definido sobre un alfabeto. Por ejemplo, el alfabeto $\Sigma = \{0, 1\}$ está formado por los símbolos 0 y 1; algunas cadenas sobre este alfabeto son 11 y 101, y un lenguaje sobre él es el conjunto de cadenas formadas por 0 y 1 que terminan en 1.

La **longitud** de una cadena w , denotada $|w|$, es la cantidad de símbolos que la componen. El símbolo ε representa la **cadena vacía**, que no tiene símbolos, de modo que $|\varepsilon| = 0$.

Dadas dos cadenas w_1 y w_2 , la **concatenación** w_1w_2 es la cadena formada por los símbolos de w_1 seguidos de los símbolos de w_2 . Dada una cadena w y un entero $n \geq 0$, por convenio w^n representa la cadena resultante de concatenar w un total de n veces, con $w^0 = \varepsilon$.

Si Σ es un alfabeto, Σ^+ denota el conjunto de todas las cadenas de uno o más símbolos que se pueden formar sobre Σ , y Σ^* denota la unión de Σ^+ y $\{\varepsilon\}$.

Dado que los lenguajes son conjuntos, las operaciones sobre conjuntos también se definen para lenguajes: unión, intersección y complemento [14]. A continuación se formalizan dos problemas que se utilizan de manera recurrente en los capítulos posteriores: determinar si una cadena pertenece a un lenguaje y determinar si un lenguaje contiene algún elemento.

Definición 1.1. El **problema de la palabra** consiste en determinar si una cadena pertenece a un lenguaje dado.

Por ejemplo, para el lenguaje L de las cadenas binarias que terminan en 0, el problema de la palabra consiste en decidir, dada una cadena w , si $w \in L$; así, $1100 \in L$ y $1101 \notin L$.

Definición 1.2. El **problema del vacío** consiste en determinar si un lenguaje es vacío.

Por ejemplo, el problema del vacío para el lenguaje formado por los números pares mayores que 5 que además son primos, consiste en decidir si tal lenguaje tiene algún elemento.

Los antecedentes del presente trabajo, que se revisan en el capítulo 2, emplean el problema del vacío para decidir la satisfacibilidad de fórmulas booleanas, problema que se presenta en la sección 1.4.

A continuación se presentan las gramáticas, un mecanismo que permite describir los elementos de un lenguaje mediante reglas de producción.

1.1.2. Gramáticas

Los lenguajes formales se pueden describir mediante **gramáticas**, un mecanismo que permite generar los elementos de un lenguaje a partir de un símbolo inicial y un conjunto de reglas de producción.

Definición 1.3. Una **gramática** es un formalismo utilizado para describir lenguajes formales. Se define como una 4-tupla

$$G = (N, \Sigma, P, S),$$

donde

- N es un conjunto finito de símbolos llamados **símbolos no terminales**;
- Σ es un conjunto finito de símbolos llamados **símbolos terminales**, que constituyen el alfabeto sobre el que se construyen las cadenas del lenguaje. Se debe cumplir que $N \cap \Sigma = \emptyset$;
- P es un conjunto finito de **reglas de producción** de la forma

$$\alpha \rightarrow \beta, \quad \text{con } \alpha \in (N \cup \Sigma)^* \setminus \Sigma^* \text{ y } \beta \in (N \cup \Sigma)^*;$$

- $S \in N$ es el **símbolo inicial** de la gramática.

Una **derivación** en la gramática consiste en seleccionar una regla de producción $\alpha \rightarrow \beta$ y sustituir una ocurrencia de α en una cadena $w \in (\Sigma \cup N)^+$ por β [14].

Por ejemplo, considere la gramática $G = (\{S\}, \{a, b\}, P, S)$ con producciones $P = \{S \rightarrow aSb, S \rightarrow ab\}$. Partiendo de S , una primera derivación aplica $S \rightarrow aSb$ y produce la cadena aSb ; una segunda derivación aplica $S \rightarrow ab$ sobre la ocurrencia de S en aSb y produce $aabb$.

Se dice que la gramática G genera una cadena $w \in \Sigma^*$ si existe una sucesión finita de derivaciones que, partiendo de S , produce w . El **lenguaje generado** por una gramática G se denota

$$L(G) = \{w \in \Sigma^* \mid S \xrightarrow{+} w\},$$

donde $\xrightarrow{+}$ indica la existencia de una sucesión finita de una o más derivaciones desde S hasta w . En el ejemplo anterior, $L(G) = \{a^n b^n \mid n \geq 1\}$.

Una vez presentadas las gramáticas, a continuación se expone la jerarquía de Chomsky.

1.1.3. Jerarquía de Chomsky

La **Jerarquía de Chomsky** [15] clasifica los lenguajes en cuatro tipos, según las restricciones en las reglas de producción de las gramáticas que los generan.

El primer tipo son las **gramáticas irrestrictas**, que no imponen restricciones sobre sus reglas de producción. Cada regla tiene la forma $\alpha \rightarrow \beta$, donde $\alpha \in (N \cup \Sigma)^* \setminus \Sigma^*$ y $\beta \in (N \cup \Sigma)^*$. Todo lenguaje generado por una gramática irrestricta se denomina **lenguaje recursivamente enumerable** [14].

El segundo tipo son las **gramáticas dependientes del contexto**. En estas, cada regla tiene la forma $\alpha A \gamma \rightarrow \alpha \beta \gamma$, donde $A \in N$, $\alpha, \beta, \gamma \in (N \cup \Sigma)^*$, y $|\beta| \geq 1$. Todo lenguaje generado por una gramática dependiente del contexto se denomina **lenguaje dependiente del contexto**. Todo lenguaje dependiente del contexto es también un lenguaje recursivamente enumerable. En [11] se demuestra que el lenguaje de las

fórmulas booleanas satisfacibles no es libre del contexto, por lo que necesariamente se sitúa en alguno de estos dos tipos superiores; este resultado se discute en el capítulo 2.

El tercer tipo son las **gramáticas libres del contexto** (CFG). En estas, cada regla tiene la forma $A \rightarrow \beta$, donde $A \in N$ y $\beta \in (N \cup \Sigma)^*$. Todo lenguaje generado por una gramática libre del contexto se denomina **lenguaje libre del contexto**. Todo lenguaje libre del contexto es también un lenguaje dependiente del contexto [14].

Las gramáticas de concatenación de rango, formalismo central del capítulo 2, se introducen como una extensión de las CFG. Además, en [10] se emplea una CFG para resolver instancias del problema de la satisfacibilidad booleana, que se presenta en la sección 1.4.

El cuarto tipo son las **gramáticas regulares**. En estas, las reglas de producción tienen la forma

$$A \rightarrow aB \quad \text{o} \quad A \rightarrow a,$$

donde $A, B \in N$ y $a \in \Sigma$.

Todo lenguaje generado por una gramática regular se denomina **lenguaje regular**. Todo lenguaje regular es un lenguaje libre del contexto. Los lenguajes regulares se relacionan con los autómatas finitos, que se presentan en la sección 1.2.

Cada tipo de la jerarquía de Chomsky agrupa los lenguajes generados por las gramáticas de ese tipo. Estas agrupaciones se denominan **clases de lenguajes**: la clase de los lenguajes regulares, la de los libres del contexto, la de los dependientes del contexto y la de los recursivamente enumerables. Por brevedad, expresiones como "los lenguajes libres del contexto" se refieren a la clase correspondiente.

Una clase de lenguajes $\mathcal{A}$ es más expresiva que una clase $\mathcal{B}$ si todo lenguaje de $\mathcal{B}$ pertenece también a $\mathcal{A}$ y existe al menos un lenguaje de $\mathcal{A}$ que no pertenece a $\mathcal{B}$. Por ejemplo, la clase de los lenguajes libres del contexto es más expresiva que la clase de los lenguajes regulares [14].

La diferencia entre las clases de lenguajes de la jerarquía de Chomsky se puede ilustrar con una familia¹ de lenguajes derivada del lenguaje Copy. Dado un alfabeto Σ , para cada entero $k \geq 1$ se define el lenguaje

$$L_{\text{copy}}^k = \{w^k \mid w \in \Sigma^*\}.$$

La familia $\{L_{\text{copy}}^k\}_{k \geq 1}$ es la que ilustra la jerarquía. El caso $k = 1$ corresponde a $L_{\text{copy}}^1 = \Sigma^*$, que es un lenguaje regular. Para $k \geq 2$, el lenguaje L_{copy}^k no es libre del contexto, aunque sí es dependiente del contexto [14]. En el capítulo 2 se emplea este lenguaje para ilustrar que las gramáticas de concatenación de rango son más expresivas que las gramáticas libres del contexto.

¹El término "familia de lenguajes" se utiliza aquí en un sentido distinto al de "clase de lenguajes" definido en el párrafo anterior: se refiere a una colección de lenguajes que se obtiene al variar un parámetro, en este caso los lenguajes L_{copy}^k al variar k , y no a una clase de la jerarquía de Chomsky.

La relación de inclusión entre las clases de la jerarquía se ilustra en la figura 1.1: una clase está contenida en otra cuando la elipse que la representa aparece dentro de la otra.

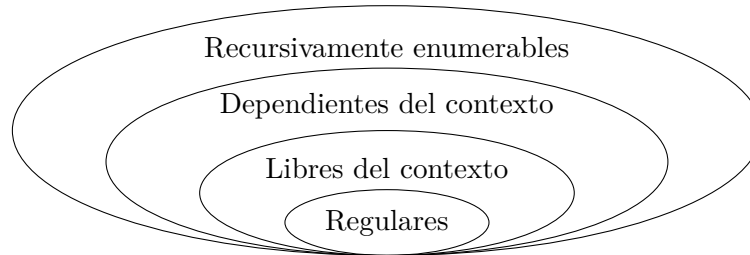


Figura 1.1: Jerarquía de Chomsky.

Cada clase de la jerarquía de Chomsky admite un modelo matemático, llamado **autómata**, que reconoce los lenguajes de la clase. En la sección 1.2 se define el **autómata finito**.

1.2. Autómatas finitos

De los modelos asociados a las clases de la jerarquía, en este trabajo solo se emplea el **autómata finito**, que reconoce los lenguajes regulares [14].

Definición 1.4. Un **autómata finito** [14] se define como una 5-tupla

$$A = (Q, \Sigma, \delta, q_0, F),$$

donde Q es un conjunto finito de **estados**, Σ es un alfabeto llamado **alfabeto de entrada**, $\delta : Q \times \Sigma \rightarrow Q$ es una **función de transición**, $q_0 \in Q$ es el **estado inicial** y $F \subseteq Q$ es el conjunto de **estados de aceptación**.

El autómata comienza en el estado inicial q_0 y lee los símbolos de la cadena uno a uno. En cada paso, la función de transición δ determina, a partir del símbolo actual y del estado actual, el estado al que pasa el autómata. Al finalizar la lectura del último símbolo, si el autómata se encuentra en un estado de F , la cadena se acepta; en caso contrario, se rechaza.

Una vez presentado el autómata finito, en la próxima sección se introducen los conceptos de complejidad computacional, que se utilizan en los capítulos posteriores para analizar la complejidad del problema del vacío en los formalismos estudiados.

1.3. Complejidad computacional

Esta sección se organiza en tres partes. Primero se presenta la notación asintótica, después se introduce la reducción polinomial. Por último se definen las clases de problemas P , NP y NP -Completo.

1.3.1. Notación asintótica

La notación asintótica permite describir el crecimiento de una función $f(n)$ a medida que n tiende a infinito. Se utiliza para clasificar la dificultad de un problema según el tiempo de ejecución de los algoritmos que lo resuelven.

Definición 1.5. Una función $g(n)$ pertenece a $O(f(n))$ si existen constantes positivas c y n_0 tales que

$$g(n) \leq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

La definición anterior expresa que $g(n)$ está acotada superiormente por un múltiplo constante de $f(n)$ para n suficientemente grande.

El tiempo de ejecución de un algoritmo es una función del tamaño n de su entrada. Se dice que un algoritmo tiene **tiempo polinomial** si esta función pertenece a $O(n^k)$ para alguna constante k . La **complejidad computacional** de un problema se define como el tiempo de ejecución, en el peor caso, del algoritmo más eficiente que lo resuelve [14].

Una vez introducida la noción de tiempo polinomial, en la próxima subsección se introduce la reducción polinomial como herramienta para relacionar la dificultad entre problemas.

1.3.2. Reducción polinomial

Los conceptos que se introducen a continuación, y los de la próxima subsección, se aplican a problemas de decisión. Un **problema de decisión** es un problema cuya respuesta para cada instancia² es sí o no.

Definición 1.6. Una **reducción polinomial** de un problema de decisión A a un problema de decisión B es una función f que se calcula en tiempo polinomial y asigna a cada entrada x de A una entrada $f(x)$ de B , tal que A responde sí sobre x si y solo si B responde sí sobre $f(x)$ [14].

La existencia de una reducción polinomial de A a B significa que B es, en términos de complejidad polinomial, al menos tan difícil como A . Como consecuencia, si B se resuelve en tiempo polinomial, A también. La reducción polinomial es la herramienta con la que se define la clase NP -Completo en la próxima subsección.

²Una instancia es una entrada del problema.

1.3.3. Clases de problemas

Los problemas computacionales [14] se agrupan en clases según el tiempo de ejecución necesario para resolverlos o verificar sus soluciones. En este trabajo se emplean las clases P , NP y NP -Completo.

Definición 1.7. Un problema pertenece a la clase P si se puede resolver en tiempo polinomial [14].

Por ejemplo, el problema 2-SAT, que se presenta en la sección 1.4.2, pertenece a P .

Definición 1.8. Un problema pertenece a la clase NP si existe un algoritmo que, dada una instancia del problema y una solución candidata³, verifica en tiempo polinomial si la solución candidata es correcta [14].

Por ejemplo, el problema SAT, que se presenta en la sección 1.4, pertenece a NP : dada una asignación de valores a las variables de una fórmula booleana, un algoritmo polinomial determina si la asignación satisface la fórmula.

Definición 1.9. Un problema pertenece a la clase NP -Completo si pertenece a NP y, además, cualquier problema en NP se puede reducir polinomialmente a él [14].

El SAT del ejemplo anterior es también NP -Completo.

La pregunta de si $P = NP$ o $P \neq NP$ permanece sin resolver y constituye uno de los problemas abiertos de la teoría de la computación [14]. El conjunto de problemas NP -Completo proporciona una base para el estudio de esta pregunta: si algún problema NP -Completo se puede resolver en tiempo polinomial, entonces todos los problemas de NP también.

Una vez introducidas estas clases, en la próxima sección se presenta el problema de la satisfacibilidad booleana, problema de decisión cuyo estudio mediante formalismos de la teoría de lenguajes motiva este trabajo.

1.4. Problema de la satisfacibilidad booleana

El problema de la satisfacibilidad booleana (SAT) es un problema de la teoría de la computación y la lógica matemática [14]. La sección se organiza en dos partes. Primero se presentan los elementos del SAT y se da su definición formal. Después se analiza el estatus del SAT como problema NP -Completo junto con sus variantes polinomiales.

³Este término también se conoce en la literatura como certificado.

1.4.1. Elementos del SAT

A continuación se presentan los elementos del SAT: variables booleanas, literales, cláusulas y fórmulas en forma normal conjuntiva.

- **Variables booleanas:** una variable booleana es una variable que puede tomar uno de dos valores: verdadero o falso.
- **Literales:** un literal es una variable booleana o su negación. Formalmente, si x es una variable booleana, entonces x y $\neg x$ son literales, donde $\neg x$ representa la negación de x . Un literal toma los valores verdadero o falso según la asignación de valores a las variables booleanas.
- **Cláusulas:** una cláusula es una disyunción, con el operador OR, de uno o más literales. Por ejemplo, la cláusula $(x \vee \neg y \vee z)$ es una disyunción de tres literales: x , $\neg y$ y z . Una cláusula es verdadera si al menos uno de sus literales es verdadero; es falsa si todos sus literales son falsos.
- **Fórmulas en forma normal conjuntiva:** una fórmula booleana en forma normal conjuntiva, abreviada CNF, es una conjunción con el operador AND de cláusulas. Por ejemplo,

$$(x \vee \neg y \vee z) \wedge (\neg x \vee y) \wedge (x \vee \neg z)$$

es una fórmula en CNF con tres cláusulas y tres variables booleanas.

Una **interpretación** de una fórmula booleana es una asignación de valores verdadero o falso a cada una de sus variables booleanas.

Una fórmula booleana es **satisfacible** si es verdadera bajo alguna de sus interpretaciones.

Definición 1.10. El problema de la **satisfacibilidad booleana**, o SAT, consiste en determinar si una fórmula booleana es satisfacible.

Toda fórmula booleana se puede transformar, en tiempo polinomial, en una fórmula en CNF que es satisfacible si y solo si la fórmula original lo es [14]. Por esta razón, en el resto del trabajo se asume sin pérdida de generalidad que toda fórmula booleana está en CNF.

Una vez definido el problema y sus elementos, a continuación se sitúa al SAT dentro de la clasificación de complejidad y se describen sus variantes polinomiales.

1.4.2. SAT como problema *NP*-Completo y variantes polinomiales

El SAT es el primer problema demostrado como *NP*-Completo y juega un rol central en la teoría de la complejidad computacional. El SAT pertenece a la clase *NP* porque, dada una asignación de valores a las variables booleanas de la fórmula, se puede verificar en tiempo polinomial si la asignación satisface la fórmula. La demostración de que el SAT es *NP*-Completo fue una de las contribuciones principales de Stephen Cook en 1971, y marcó el inicio de la teoría de la *NP*-Complejidad [8].

Un SAT en el que cada cláusula tiene exactamente n literales distintos se denomina n -SAT. Para el problema 2-SAT existe una solución polinomial que determina si la fórmula booleana es satisfacible o no [13]. Para el problema 3-SAT no se conoce ningún algoritmo polinomial que permita determinar si una fórmula booleana es satisfacible o no [14]. Toda fórmula booleana del problema n -SAT se puede reducir a una fórmula del problema 3-SAT que es satisfacible si y solo si la fórmula original lo es, por lo que SAT y 3-SAT son equivalentes en términos de complejidad computacional [14].

Aunque no se conoce algoritmo polinomial para resolver el SAT general, existen variantes particulares que sí admiten solución en tiempo polinomial, como 2-SAT, Horn-SAT y XOR-SAT.

El problema **2-SAT** es una variante del SAT donde cada cláusula contiene exactamente dos literales. El problema 2-SAT admite solución en tiempo polinomial mediante una modelación basada en grafos, con algoritmos como la detección de componentes fuertemente conexas en el grafo de implicación [13].

El problema **Horn-SAT** es una variante del SAT donde cada cláusula tiene a lo sumo un literal positivo. El problema Horn-SAT admite solución en tiempo polinomial mediante el algoritmo de resolución de Horn [9].

El problema **XOR-SAT** es una variante del SAT donde cada cláusula representa una operación XOR sobre sus literales. El problema XOR-SAT admite solución en tiempo polinomial al transformarlo en un sistema de ecuaciones lineales módulo 2 y aplicar eliminación de Gauss [17].

Las variantes anteriores se obtienen al restringir la forma de las cláusulas. Otra forma de ampliar las subclases del SAT con solución en tiempo polinomial consiste en modelar el problema mediante formalismos de la teoría de lenguajes. En este capítulo se introdujeron las nociones en que se apoya ese enfoque: la teoría de lenguajes, los autómatas finitos, la complejidad computacional y el problema SAT. En el próximo capítulo se presentan las gramáticas de concatenación de rango y los antecedentes en el estudio del SAT con teoría de lenguajes.

Capítulo 2

Gramáticas de concatenación de rango y el SAT como lenguaje formal

Las gramáticas de concatenación de rango son el formalismo sobre el que se desarrolla el resto del trabajo. El capítulo se organiza en tres secciones. La sección 2.1 introduce las gramáticas de concatenación de rango, su presentación intuitiva, sus definiciones formales y un ejemplo de reconocimiento. La sección 2.2 expone las propiedades de clausura del formalismo y su caracterización en términos de complejidad. La sección 2.3 revisa los antecedentes directos del trabajo: los estudios previos de la Facultad de Matemática y Computación de la Universidad de La Habana que abordan el SAT mediante formalismos de la teoría de lenguajes.

2.1. Gramáticas de concatenación de rango

Las gramáticas de concatenación de rango (RCG) [6] son un formalismo propuesto en 1998 por Pierre Boullier, un investigador en el campo de la lingüística computacional. El objetivo inicial del formalismo era proporcionar un modelo más expresivo que las gramáticas libres del contexto para describir ciertos fenómenos del lenguaje natural, como los números chinos y el orden variable de algunas palabras en alemán [5].

Antes de entrar a la presentación del formalismo, es necesario precisar una convención terminológica que rige a lo largo de todo el documento. El formalismo que se presenta en este capítulo corresponde, en la literatura, a las **gramáticas de concatenación de rango positivas** (pRCG), una de las dos variantes del formalismo general de Boullier. La otra variante, las gramáticas de concatenación de rango con negación, admite cláusulas con predicados negados y no se utiliza en este trabajo. La restricción a la variante positiva es importante porque, como se establece más

adelante, los lenguajes que se reconocen por una pRCG son exactamente aquellos cuyo problema de la palabra está en P , y los resultados de los capítulos posteriores se apoyan en dicha caracterización. Para abreviar, a lo largo del documento se adopta el nombre **gramática de concatenación de rango** (RCG) para referirse, salvo mención explícita, a la variante positiva.

2.1.1. Presentación intuitiva

A continuación se introducen las nociones de predicado, argumento, sustitución de rango y derivación, que se utilizan más adelante.

A los no terminales de una RCG se les llama **predicados**. Cada predicado tiene un conjunto de argumentos. A la cantidad de argumentos de un predicado se le denomina **aridad**. Por ejemplo, $A(X, Y)$ denota el predicado A con argumentos X e Y ; en este caso, la aridad de A es 2.

Cada argumento de un predicado puede estar formado por variables y terminales. Por convenio, las variables se denotan por letras mayúsculas del final del alfabeto, y los terminales con letras minúsculas. Con este convenio, el predicado $B(aX, XY, abZ)$ tiene aridad 3. El primer argumento está formado por el terminal a y la variable X . El segundo argumento está formado por la concatenación de las variables X e Y . El tercer argumento está formado por la concatenación de los terminales a y b y la variable Z .

Cada predicado reconoce un vector de cadenas cuya dimensión es la aridad del predicado. Cada cadena del vector se asocia a un argumento del predicado.

Por ejemplo, si al predicado $A(X, Y)$ se le asocia el vector $[abc, d]$, al primer argumento de A se le asigna la cadena abc y al segundo la cadena d . Esto significa que $X = abc$ y que $Y = d$.

De manera similar, si al predicado $B(aX, XY, abZ)$ se le asigna el vector $[aa, de, abcc]$, al primer argumento de B se le asigna la cadena aa , al segundo la cadena de y al tercero la cadena $abcc$. Es decir, $aX = aa$, $XY = de$ y $abZ = abcc$. Como X es una variable y se tiene que $aX = aa$, el único valor que puede tomar X es $X = a$. En el tercer argumento ocurre algo similar: como $abZ = abcc$, para que se cumpla la igualdad el único valor posible para Z es $Z = cc$. En el segundo argumento, las variables X e Y deben tomar valores tales que su concatenación sea de ; esto se puede hacer de tres formas: $X = d$, $Y = e$; $X = de$, $Y = \varepsilon$; y $X = \varepsilon$, $Y = de$.

Si al predicado $B(aX, XY, abZ)$ se le asigna el vector $[bb, de, abcc]$, entonces al tenerse $aX = bb$, X no puede tomar ningún valor: el primer carácter b de la cadena no coincide con el terminal a que encabeza el argumento aX . Por tanto, el predicado B no reconoce este vector.

A las producciones de esta gramática se les denomina **cláusulas**. El lado izquierdo de la producción siempre está formado por un único predicado. El lado derecho puede

contener cualquier cantidad de predicados con sus respectivos argumentos, o la cadena vacía. Un ejemplo de cláusula es

$$A(XYZ, W) \rightarrow B(X)C(XY, Z)D(W).$$

La regla anterior tiene el siguiente significado: el predicado A recibe un vector de dimensión 2; a partir de las cadenas del vector, se asignan valores a las variables X , Y , Z y W , y con esos valores se construyen los vectores que reciben los predicados B , C y D .

El proceso se ilustra con un ejemplo en el que el predicado A recibe el vector de cadenas $[abc, d]$. El primer paso es asignar las cadenas del vector a los argumentos del predicado. El primer argumento de A es abc y el segundo es d . Como el primer argumento de A está definido como XYZ y el segundo como W , debe cumplirse que $XYZ = abc$ y $W = d$. Esto significa que las variables X , Y y Z deben tomar los valores posibles cuya concatenación sea abc , mientras que W solo puede tomar el valor d . En el cuadro 2.1 se muestran algunas asignaciones posibles para las variables X , Y , Z y W a partir del vector de entrada.

Tabla 2.1: Asignaciones posibles para las variables X , Y , Z y W a partir del vector $[abc, d]$.

X	Y	Z	W
ε	ε	abc	d
ε	a	bc	d
ε	ab	c	d
ε	abc	ε	d
a	ε	bc	d
a	b	c	d
a	bc	ε	d
ab	ε	c	d
ab	c	ε	d
abc	ε	ε	d

Continuando con el ejemplo, se supone que los valores de X , Y y Z son $X = ab$, $Y = \varepsilon$ y $Z = c$. A partir de esta asignación se construyen los vectores con los que se evalúan los predicados del lado derecho $B(X)C(XY, Z)D(W)$. Como $X = ab$, $Y = \varepsilon$, $Z = c$ y $W = d$, el lado derecho se evalúa con los vectores $[ab]$, $[ab, c]$ y $[d]$, y el resultado es la derivación $B(ab)C(ab, c)D(d)$.

El proceso de asignar los vectores a los argumentos, identificar los valores de las variables e instanciar los lados derechos se repite en cada uno de los predicados del lado derecho de la cláusula.

Existe otro tipo de producciones, denominadas cláusulas terminales, en las que un predicado deriva en ε para determinados valores de sus argumentos. Por ejemplo,

$$B(ab) \rightarrow \varepsilon, \quad C(ab, c) \rightarrow \varepsilon, \quad D(d) \rightarrow \varepsilon.$$

La forma general de estas producciones es $A(x_1, \dots, x_k) \rightarrow \varepsilon$, donde k es la aridad de A y cada x_i es un argumento de A , esto es, una cadena de variables y terminales.

Un predicado reconoce un vector de cadenas si existe una asignación de sus argumentos a valores de las variables y una secuencia de derivaciones que termina en la cadena vacía. Por ejemplo, en el caso de $B(ab) \rightarrow \varepsilon$, el predicado B reconoce la cadena ab .

A modo de ilustración, se considera el siguiente conjunto de producciones:

1. $A(XYZ, W) \rightarrow B(X)C(XY, Z)D(W)$,
2. $B(ab) \rightarrow \varepsilon$,
3. $C(ab, c) \rightarrow \varepsilon$,
4. $D(d) \rightarrow \varepsilon$.

Con estas producciones, el predicado A reconoce el vector $[abc, d]$: con la asignación $X = ab$, $Y = \varepsilon$, $Z = c$ y $W = d$, el predicado A deriva en $B(ab)C(ab, c)D(d)$, y cada uno de los predicados $B(ab)$, $C(ab, c)$ y $D(d)$ deriva en la cadena vacía.

Un predicado no reconoce un vector de cadenas cuando para ninguna asignación de variables se deriva en la cadena vacía.

Presentadas estas nociones, a continuación se introducen las definiciones formales de las gramáticas de concatenación de rango.

2.1.2. Definiciones formales

El formalismo toma su nombre de la noción de rango, que identifica las subcadenas asignadas a las variables.

Definición 2.1. Un **rango** de una cadena es un par (i, j) de posiciones, con i y j enteros no negativos tales que $i \leq j$, que delimita la subcadena comprendida entre la posición i y la posición j .

Por ejemplo, con las posiciones indexadas desde 0, para la cadena $abcd$ el rango $(1, 3)$ delimita la subcadena bc .

Definición 2.2. Una **gramática de concatenación de rango** se define como una 5-tupla

$$G = (N, T, V, P, S),$$

donde:

- N es un conjunto finito de **predicados** o símbolos no terminales; cada predicado tiene una aridad, que indica la dimensión del vector de cadenas que reconoce;
- T es un conjunto finito de **símbolos terminales**;
- V es un conjunto finito de **variables**;
- P es un conjunto finito de **cláusulas** de la forma

$$A(x_1, x_2, \dots, x_k) \rightarrow B_1(y_{1,1}, y_{1,2}, \dots, y_{1,m_1}) \dots B_n(y_{n,1}, y_{n,2}, \dots, y_{n,m_n}),$$

donde $A, B_i \in N$, $x_i, y_{i,j} \in (V \cup T)^*$, $n \geq 0$, k es la aridad de A y m_i es la aridad de B_i ; cuando $n = 0$, el lado derecho es la cadena vacía y la cláusula se denomina terminal;

- $S \in N$ es el **predicado inicial** de la gramática, que siempre tiene aridad 1.

Por ejemplo, a continuación se muestra una gramática de concatenación de rango que reconoce el lenguaje $L_{\text{copy}}^3 = \{w^3 \mid w \in \{a, b, c\}^*\}$:

$$G_{\text{copy}}^3 = (N, T, V, P, S),$$

donde $N = \{A, S\}$, $T = \{a, b, c\}$, $V = \{X, Y, Z\}$, el símbolo inicial es S y el conjunto de cláusulas P es el siguiente:

1. $S(XYZ) \rightarrow A(X, Y, Z)$,
2. $A(aX, aY, aZ) \rightarrow A(X, Y, Z)$,
3. $A(bX, bY, bZ) \rightarrow A(X, Y, Z)$,
4. $A(cX, cY, cZ) \rightarrow A(X, Y, Z)$,
5. $A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon$.

A diferencia de las gramáticas presentadas en el capítulo anterior, que se definen a partir de un mecanismo de generación de cadenas, las RCG se formulan directamente como un mecanismo de reconocimiento. Su funcionamiento consiste en determinar si una cadena pertenece o no al lenguaje.

Definición 2.3. Una **sustitución de rango** es un mecanismo que asigna a cada variable un rango de la cadena que se reconoce, de acuerdo con la estructura del argumento. La variable se reemplaza entonces por la subcadena que ese rango delimita.

Por ejemplo, dado el predicado $A(Xa)$ con $X \in V$ y $a \in T$, la estructura del argumento de A es una variable X seguida del terminal a . Si el predicado A recibe la cadena baa , entonces X se puede asociar con la subcadena ba de la cadena original: si $X = ba$, entonces $Xa = baa$. La variable X no puede tomar el valor baa , porque ningún carácter de la cadena de entrada coincide con el terminal a al final del argumento. La variable X tampoco puede tomar el valor b , porque con ese valor el argumento Xa no cubre la cadena completa.

Definición 2.4. El **tamaño** de una RCG $G = (N, T, V, P, S)$, que se denota $|G|$, es

$$|G| = |N| + |T| + |P|,$$

la suma de las cantidades de predicados, símbolos terminales y cláusulas.

Definición 2.5. Las **gramáticas de concatenación de rango simple** (sRCG) son un subconjunto de las RCG que restringen la forma de las cláusulas. Una RCG G es **simple** si los argumentos del lado derecho de cada cláusula son variables distintas, y todas estas variables aparecen una sola vez en los argumentos del lado izquierdo.

Por ejemplo, la cláusula $A(XY) \rightarrow B(X)C(Y)$ cumple la condición de simplicidad: los argumentos del lado derecho, X e Y , son variables distintas, y cada una aparece una sola vez en el lado izquierdo. La cláusula $A(X) \rightarrow B(X)C(X)$ no la cumple, porque la variable X aparece en dos argumentos del lado derecho; una RCG que la contenga no es simple.

Las sRCG son equivalentes a los sistemas lineales de reescritura libres del contexto (LCFRS) introducidos en [18]. Este subconjunto se utiliza en [1] para una variante del SAT denominada **Satisfacibilidad Booleana de Concatenación de Rango Simple**, que se discute en la sección 2.3.

Definición 2.6. Una RCG es **no lineal** si una misma variable aparece más de una vez en el lado derecho de una cláusula.

Por ejemplo, la cláusula $A(X) \rightarrow B(X)C(X)$ es no lineal, pues la variable X aparece en dos argumentos del lado derecho.

Una vez introducidas las definiciones formales, a continuación se presenta un ejemplo completo de reconocimiento de una cadena por una RCG, aplicando los mecanismos descritos en esta sección.

2.1.3. Ejemplo de reconocimiento

La presentación intuitiva describió cómo se asocian los elementos de un vector a los argumentos de un predicado y cómo se construyen los vectores que reciben los predicados del lado derecho de una cláusula. En esta subsección se aplica ese proceso

a un caso completo: el reconocimiento de la cadena $abcabcabc$ por la gramática G_{copy}^3 presentada anteriormente.

La cadena $abcabcabc$ se reconoce por G_{copy}^3 mediante la siguiente secuencia de derivaciones:

$$S(abcabcabc) \rightarrow A(abc, abc, abc) \rightarrow A(bc, bc, bc) \rightarrow A(c, c, c) \rightarrow A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon.$$

A continuación se describen los pasos.

- **Primer paso:** en la primera cláusula $S(XYZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = abc, Y = abc$ y $Z = abc$. De esta forma se deriva en el predicado $A(abc, abc, abc)$.
- **Segundo paso:** en la segunda cláusula $A(aX, aY, aZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = bc, Y = bc$ y $Z = bc$. Con estos valores se deriva en el predicado $A(bc, bc, bc)$.
- **Tercer paso:** en la tercera cláusula $A(bX, bY, bZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = c, Y = c$ y $Z = c$. Con estos valores se deriva en el predicado $A(c, c, c)$.
- **Cuarto paso:** en la cuarta cláusula $A(cX, cY, cZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = \varepsilon, Y = \varepsilon$ y $Z = \varepsilon$. Con estos valores se deriva en el predicado $A(\varepsilon, \varepsilon, \varepsilon)$.
- **Quinto paso:** en la última cláusula $A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon$ se deriva en la cadena vacía, con lo que la cadena $abcabcabc$ queda reconocida.

Una vez presentado el proceso de derivación, en la siguiente sección se analizan las propiedades del formalismo: clausura bajo intersección, comparación con las gramáticas libres del contexto y complejidad del problema de la palabra.

2.2. Propiedades de las RCG

Las RCG son cerradas bajo unión, intersección y complemento [6]. De estas tres operaciones, la clausura bajo intersección es la que importa para este trabajo. En el enfoque que se revisa en la sección 2.3, la satisfacibilidad de una fórmula se decide según si una intersección de dos lenguajes es vacía. En esta sección se presentan estas propiedades de clausura y la caracterización del reconocimiento en términos de complejidad. La clausura bajo intersección se usa en el capítulo 5, donde se construye una RCG cuyo lenguaje es esa intersección. El reconocimiento polinomial se usa en el capítulo 3, donde se caracteriza la decisión de su vacío.

Teorema 2.1 (Clausura bajo intersección). *Si L_1 y L_2 son lenguajes reconocidos por RCG, entonces $L_1 \cap L_2$ también lo es [6].*

Las RCG son más expresivas que las gramáticas libres del contexto. Por ejemplo, para todo $k \geq 2$, el lenguaje $L_{\text{copy}}^k = \{w^k \mid w \in \Sigma^*\}$ es dependiente del contexto y no libre del contexto [14]. Aun así, se reconoce por una RCG. En particular, el lenguaje L_{copy}^3 se reconoce por la gramática G_{copy}^3 del ejemplo anterior.

La próxima subsección analiza el problema de la palabra de las RCG y su complejidad.

2.2.1. Complejidad y caracterización en términos de P

El siguiente resultado es una caracterización del reconocimiento de las RCG en términos de complejidad.

Teorema 2.2 (Caracterización en términos de P). *Un lenguaje se reconoce por una RCG en su variante positiva si y solo si su problema de la palabra está en P [4, 7].*

La dirección que afirma que todo lenguaje que se reconoce por una RCG tiene su problema de la palabra en P se sigue de la existencia de un algoritmo de reconocimiento polinomial para el formalismo, que se describe a continuación.

El algoritmo de reconocimiento estándar para las RCG utiliza un proceso de memorización sobre las cadenas asignadas a los argumentos de los predicados [6]. La cantidad máxima de rangos de una cadena de longitud n es $O(n^2)$, y la máxima aridad de un predicado es constante, por lo que el proceso de memorización emplea una cantidad polinomial de estados. La complejidad del algoritmo es $O(|P|n^{2h(l+1)})$, donde h es la máxima aridad de un predicado, l es la máxima cantidad de predicados en el lado derecho de una cláusula y n es la longitud de la cadena que se reconoce. El teorema establece, además, la dirección inversa: todo lenguaje cuyo problema de la palabra está en P se reconoce por una RCG.

El alcance exacto de la caracterización y sus consecuencias para la relación entre formalismos gramaticales y el SAT son el objeto central de los capítulos posteriores.

Una vez establecida la caracterización de las RCG como formalismo que reconoce exactamente los lenguajes cuyo problema de la palabra está en P , en la siguiente sección se revisan los antecedentes del trabajo.

2.3. Antecedentes en el estudio del SAT mediante formalismos de la teoría de lenguajes

El estudio del problema de la satisfacibilidad booleana mediante formalismos de la teoría de lenguajes cuenta con antecedentes en la Facultad de Matemática y Compu-

tación de la Universidad de La Habana. El presente trabajo se apoya en dos de ellos: el problema de la satisfacibilidad booleana libre del contexto [10], abreviado SATCF, y el de la satisfacibilidad booleana de concatenación de rango simple [1], abreviado SATSRC.

SATCF y SATSRC comparten una misma estrategia. A una fórmula booleana se le asocia un lenguaje regular que representa sus interpretaciones verdaderas, bajo el supuesto de que cada ocurrencia de variable booleana se trata por separado. Sobre ese lenguaje se impone después que las ocurrencias de una misma variable booleana tomen el mismo valor, mediante una gramática. La satisfacibilidad se decide según si la intersección de ambos lenguajes es vacía.

La sección se organiza según esa estrategia. En la subsección 2.3.1 se describe el autómata con que se construye el lenguaje regular de las interpretaciones. En la subsección 2.3.2 se presentan la gramática que impone la consistencia, la clase de fórmulas que cada trabajo resuelve y la razón por la que las RCG son el siguiente formalismo de esta línea de trabajo.

2.3.1. El autómata de las interpretaciones

El lenguaje regular de las interpretaciones se construye mediante un autómata finito, el **autómata booleano** [10]. Su alfabeto de entrada es $\{0,1\}$, cuyos símbolos son los valores de verdad: 1 el verdadero y 0 el falso. Cada ocurrencia de variable booleana de la fórmula aporta uno de esos símbolos, según su valor. Una cadena de ceros y unos describe así una asignación a las ocurrencias, tratadas por separado, y se acepta cuando esa asignación hace verdadera la fórmula.

El autómata de una sola ocurrencia consta de un estado inicial q_0 y dos estados más, V y F . Al leer 1 se pasa a V y al leer 0 se pasa a F . El estado V es de aceptación y representa que la fórmula resulta verdadera; el estado F representa lo contrario. La figura 2.1 lo muestra.

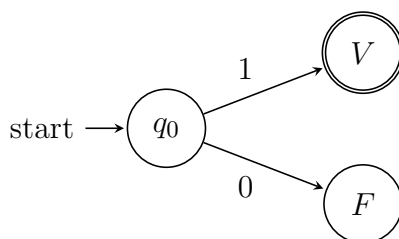


Figura 2.1: Autómata booleano de una ocurrencia de variable.

Los autómatas de las ocurrencias se combinan según los operadores de la fórmula [10]. El autómata de una fórmula compuesta se obtiene de los autómatas de sus subfórmulas, según el operador principal.

La negación se aplica a una subfórmula. Sea A una subfórmula y B su autómata booleano; con la negación se intercambian el estado de aceptación V y el de rechazo F , de modo que el autómata de $\neg A$ es B con V como no aceptante y F como aceptante. Cada cadena que hace verdadera a A hace falsa a $\neg A$, y al revés. La figura 2.2 lo muestra.

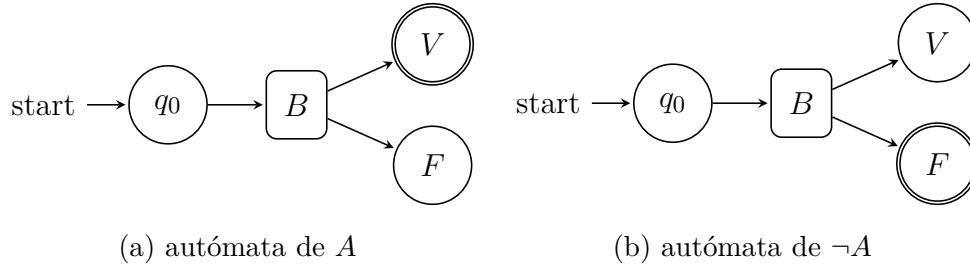


Figura 2.2: La negación intercambia el papel de V y F : en $\neg A$ el estado aceptante es F .

La conjunción y la disyunción se aplican a dos subfórmulas. Sean A_1 y A_2 dos subfórmulas y B_1, B_2 sus autómatas booleanos, con estados de aceptación V_1, V_2 y de rechazo F_1, F_2 .

En la disyunción $A_1 \vee A_2$ se concatenan los autómatas de A_1 y de A_2 . Si en el autómata de A_1 se alcanza el estado V_1 , la disyunción ya resulta verdadera; desde ahí, a lo largo de un **camino de extensión**¹, se consumen los símbolos de A_2 hasta el estado V del autómata combinado. Si se alcanza el estado F_1 , el valor depende de A_2 , y por eso el estado F_1 se une con el estado inicial del autómata de A_2 ; los estados V_2 y F_2 pasan a ser los del autómata combinado. La figura 2.3 lo muestra.

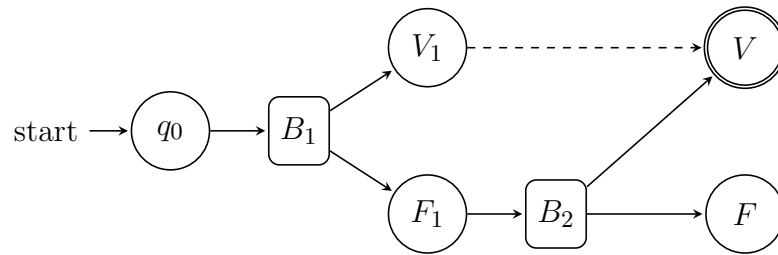


Figura 2.3: Disyunción $A_1 \vee A_2$. Al alcanzar V_1 , un camino de extensión (flecha discontinua) lleva al estado aceptante V ; al alcanzar F_1 , el cómputo continúa en B_2 .

La conjunción $A_1 \wedge A_2$ se construye de forma análoga a la disyunción, con los papeles de V y F intercambiados. Si en el autómata de A_1 se alcanza el estado F_1 , la

¹Una sucesión de estados en la que se consumen los símbolos restantes de la cadena y se llega al estado ya determinado, cualquiera que sea su valor.

conjunción ya resulta falsa, y a lo largo de un camino de extensión se consumen los símbolos de A_2 hasta el estado F del autómata combinado. Si se alcanza el estado V_1 , el valor depende de A_2 , y el estado V_1 se une con el estado inicial del autómata de A_2 . La figura 2.4 lo muestra.

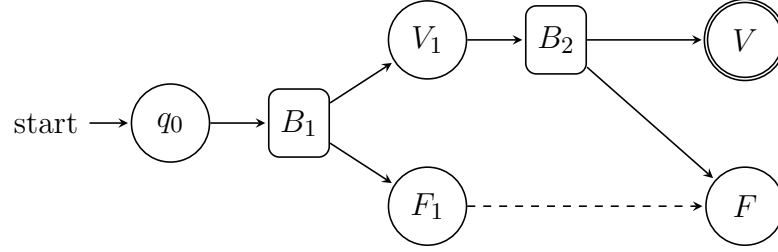


Figura 2.4: Conjunción $A_1 \wedge A_2$. Al alcanzar F_1 , un camino de extensión (flecha discontinua) lleva al estado F ; al alcanzar V_1 , el cómputo continúa en B_2 .

Por ejemplo, para la fórmula $A = x_1 \wedge (x_2 \vee x_1)$, cuya secuencia de ocurrencias es $x_1 x_2 x_1$, se construye primero el autómata de cada subfórmula, x_1 y $x_2 \vee x_1$. El de x_1 es el de una sola ocurrencia; el de $x_2 \vee x_1$ se obtiene por la regla de la disyunción. La figura 2.5 muestra ambos.

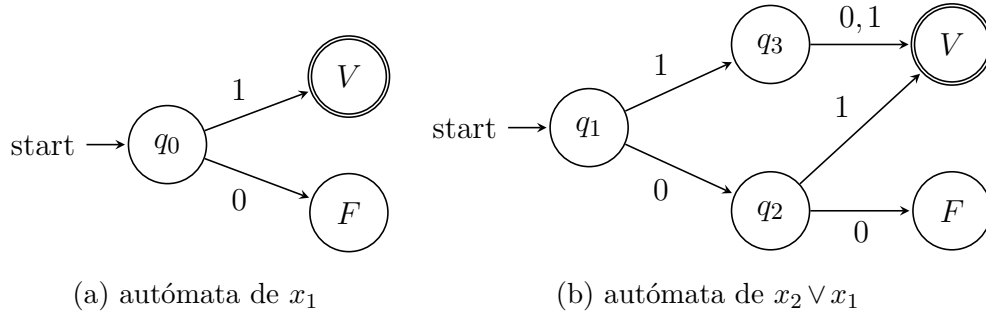


Figura 2.5: Autómatas de x_1 y de $x_2 \vee x_1$, las dos subfórmulas de A .

Al combinar los dos por la regla de la conjunción se obtiene el autómata de A : desde la rama verdadera de x_1 se entra en el autómata de $x_2 \vee x_1$, y desde la rama falsa se llega al estado F por un camino de extensión. La figura 2.6 lo muestra.

Un camino del estado inicial a V corresponde a una asignación que hace verdadera la fórmula cuando cada ocurrencia se trata por separado. Por ejemplo, con la cadena 110 se llega del estado inicial a V ; en ella se asigna 1 a la primera ocurrencia de x_1 , 1 a x_2 y 0 a la segunda ocurrencia de x_1 , valores con los que $x_1 \wedge (x_2 \vee x_1)$ resulta verdadera. En esa misma cadena, las dos ocurrencias de x_1 no toman el mismo valor, algo que no ocurre en ninguna interpretación. El autómata booleano de una fórmula con n ocurrencias de variable booleana tiene $O(n^2)$ estados [10]. En él todas

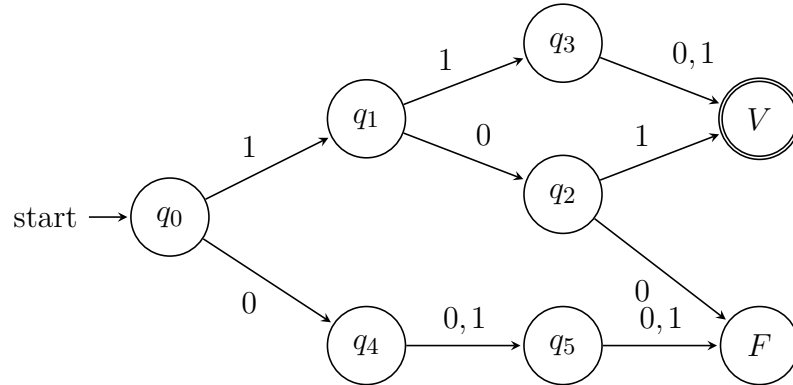


Figura 2.6: Autómata booleano de la fórmula $A = x_1 \wedge (x_2 \vee x_1)$. El estado V acepta las cadenas que hacen verdadera la fórmula cuando cada ocurrencia se trata por separado.

las ocurrencias se tratan por separado, incluso las de una misma variable booleana, y por eso hace falta un segundo mecanismo que imponga la consistencia entre las ocurrencias.

La próxima subsección presenta la gramática que impone esa consistencia y la clase de fórmulas que cada trabajo resuelve con ella.

2.3.2. La consistencia entre ocurrencias y la clase de fórmulas

Una cadena de ceros y unos, con un símbolo por ocurrencia, es **consistente** si en las posiciones de cada variable booleana lleva siempre el mismo valor. La gramática genera las cadenas consistentes de la longitud dada, y la intersección de ese lenguaje con el del autómata booleano contiene las interpretaciones verdaderas de la fórmula. La satisfacibilidad equivale entonces a que esa intersección no sea vacía. SATCF y SATSRC se diferencian en el formalismo de esa gramática, y con ello en la clase de fórmulas que resuelven.

En SATCF la consistencia se impone con una gramática libre del contexto, equivalente a un autómata de pila [10]. En la pila del autómata se guarda el valor de una variable booleana en su primera ocurrencia y se compara en las siguientes. Como en una pila se accede a los valores en orden inverso al de guardado, con este esquema la consistencia solo se verifica cuando las ocurrencias están bien anidadas: si entre dos ocurrencias consecutivas de una variable booleana aparece una ocurrencia de otra, todas las ocurrencias de esa otra deben caer entre las dos primeras. Una secuencia de ocurrencias con esa propiedad tiene un **orden libre del contexto** [10]. Por ejemplo, en la secuencia $x_1 x_2 x_2 x_1$ las dos ocurrencias de x_2 caen entre las de x_1 ; la figura 2.7(a) muestra ese anidamiento. Con SATCF se resuelven en tiempo $O(n^3)$ las

fórmulas cuyo orden es de ese tipo.

En SATSRC la consistencia se impone con una gramática de concatenación de rango simple [1]. Frente a la pila, que sigue un solo fragmento de la cadena, una sRCG admite varios a la vez, por lo que admite cruces entre ocurrencias que el orden libre del contexto excluye. Una secuencia para la que existe una sRCG de tamaño polinomial que genera sus cadenas consistentes tiene un **orden de concatenación de rango simple** [1]. Toda secuencia con orden libre del contexto lo tiene, y además lo tienen secuencias con cruces. Por ejemplo, en la secuencia $x_2 x_1 x_3 x_2 x_1$ las ocurrencias de x_2 y las de x_1 se cruzan, de modo que ese orden no es libre del contexto pero sí de concatenación de rango simple. La figura 2.7(b) muestra ese cruce. Con SATSRC se resuelven esas fórmulas en tiempo $O(n^k)$, donde k es la aridad de la sRCG, así que el grado del polinomio crece con la aridad necesaria para el cruce.

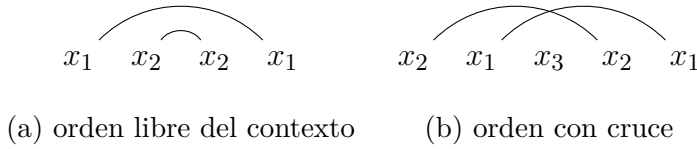


Figura 2.7: Arcos que unen las ocurrencias de cada variable booleana. En (a) los arcos se anidan; en (b), los de x_2 y los de x_1 se cruzan.

Una parte de las fórmulas ya se resuelve en tiempo polinomial. Las que no tienen cruces entre ocurrencias tienen orden libre del contexto y se deciden con SATCF; al aparecer cruces, la decisión sigue siendo polinomial con SATSRC mientras la aridad necesaria esté acotada. El problema empieza cuando no lo está: el exponente de la cota $O(n^k)$ crece con la cantidad de ocurrencias y la cota deja de ser polinomial.

Para codificar la consistencia de esas fórmulas hace falta la no linealidad. En una sRCG cada variable aparece una sola vez, de modo que, para igualar dos ocurrencias, hay que seguirlas en argumentos separados de la cláusula, y la cantidad de argumentos, esto es, la aridad, crece con el cruce. Con la no linealidad esa igualdad se expresa de forma directa: una misma variable aparece más de una vez en el lado derecho de una cláusula, y las subcadenas en que aparece deben coincidir, sin que la aridad tenga que crecer.

La no linealidad es justo la diferencia entre una sRCG y una RCG. Sin ella se reconocen los mismos lenguajes que con una sRCG, así que no se obtiene ninguna fórmula nueva. Por eso la siguiente clase natural de esta línea son las RCG.²

La clase es cerrada bajo intersección y el reconocimiento de la consistencia con una RCG sigue siendo polinomial, según los teoremas 2.1 y 2.2; pero el SAT no se decide

²Entre las sRCG y las RCG está pMCFG, con una forma restringida de no linealidad. Este trabajo no la aborda: ya existe un trabajo de la Facultad en esa dirección [2].

por el reconocimiento, sino por el vacío de la intersección. En SATCF y SATSRC esa decisión es polinomial porque el vacío de las CFG y las sRCG lo es, no por una estrategia particular. Con la no linealidad de las RCG la aridad ya no crece con el cruce, pero el vacío de la intersección deja de ser polinomial, y ahí queda concentrada la dificultad.

En este capítulo se presentaron las gramáticas de concatenación de rango, con su reconocimiento polinomial y su clausura bajo intersección, y el esquema por intersección con que se decide la satisfacibilidad en SATCF y SATSRC. En el próximo capítulo se estudia ese vacío: se construye un procedimiento explícito de decisión y se caracteriza su costo.

Capítulo 3

Vacío de la intersección de una RCG con un lenguaje regular finito

El esquema por intersección del capítulo 2 reduce la satisfacibilidad de una fórmula a decidir si la intersección de dos lenguajes es vacía [10, 1].

El paso de las gramáticas libres del contexto y de las gramáticas de concatenación de rango simples a las RCG no conserva de manera automática la tratabilidad de ese enfoque. Bertsch y Nederhof prueban que, para RCG, el vacío de la intersección con un lenguaje regular arbitrario es indecidible. También prueban que, si el lenguaje regular es finito y se representa mediante un autómata acíclico, el problema de decidir si la intersección es no vacía es NP-completo [4]. En el presente capítulo se da un algoritmo de fuerza bruta para ese último caso.

Este algoritmo no sustituye el resultado de complejidad de Bertsch y Nederhof. Se precisa qué procedimiento de decisión se obtiene cuando el autómata es acíclico y se demuestra su correctitud. Su valor está en exponer qué ocurre al decidir el vacío con los dos formalismos por separado y en caracterizar su costo. Con él se decide el problema, pero no se obtiene una representación de la intersección que se pueda estudiar más allá de las propiedades del autómata. Por eso en el capítulo 5 se construye un objeto que, en un solo formalismo, expresa de manera simbólica¹ lo que reconocen ambos.

El capítulo se organiza de la siguiente manera. En la sección 3.1 se presentan los lenguajes regulares finitos. En la sección 3.2 se formalizan los autómatas acíclicos y sus caminos aceptantes. En la sección 3.3 se define el problema de vacío de la intersección. Por último, en la sección 3.4 se presenta el algoritmo de decisión, se demuestra su correctitud y se analiza su complejidad.

¹Es decir, mediante la gramática misma, y no recorriendo los dos formalismos por separado.

3.1. Lenguajes regulares finitos

El enfoque SAT por intersección trabaja con un lenguaje regular que representa interpretaciones de una fórmula. Para una fórmula con una cantidad finita de ocurrencias de variables booleanas, ese lenguaje es finito. La presente sección fija la noción de lenguaje regular finito que será usada en el resto del capítulo.

Definición 3.1. Un lenguaje L es **finito** si existe un entero $k \geq 0$ tal que L tiene exactamente k cadenas.

Definición 3.2. Un **lenguaje regular finito** es un lenguaje regular que también es finito.

Una familia de lenguajes finitos puede tener una representación mediante autómatas de tamaño polinomial y contener, al mismo tiempo, una cantidad exponencial de cadenas.

Por ejemplo, para cada $n \geq 1$, sea

$$L_n = \{0,1\}^n.$$

El lenguaje L_n es finito y contiene 2^n cadenas. No obstante, existe un autómata finito con $n+1$ estados que reconoce L_n : el autómata avanza de un estado al siguiente al leer 0 o 1, y acepta solo después de leer exactamente n símbolos.

El ejemplo anterior muestra por qué no basta con saber que el lenguaje regular es finito. La finitud garantiza que una enumeración termina, pero no garantiza que termine en tiempo polinomial respecto al tamaño de la representación. Cuando un lenguaje regular es finito, admite una representación mediante un autómata acíclico, que se define en la sección siguiente.

3.2. Autómatas acíclicos

En el presente capítulo se usa el autómata finito del capítulo 1, con una diferencia: la función de transición es parcial, no total, es decir, $\delta(q,a)$ puede no estar definida para algunos pares (q,a) . Cuando $\delta(q,a)$ no está definida, la lectura del símbolo a en el estado q no conduce a ningún estado.

Definición 3.3. Sean $p_0, p_1, \dots, p_m$ estados de Q y $a_1, a_2, \dots, a_m$ símbolos de Σ , con $m \geq 0$. Un **camino** desde p_0 hasta p_m es una secuencia

$$\pi = p_0 a_1 p_1 a_2 p_2 \cdots a_m p_m$$

tal que, para cada $1 \leq i \leq m$, se cumple

$$\delta(p_{i-1}, a_i) = p_i.$$

La cadena que se lee a lo largo de π , denotada $w(\pi)$, es $a_1a_2\cdots a_m$. Si $m = 0$, el camino no contiene transiciones y la cadena que se lee es ε .

Definición 3.4. Un camino π en el autómata es **aceptante** si empieza en el estado inicial q_0 y termina en un estado de aceptación. El lenguaje aceptado por A es

$$L(A) = \{w(\pi) \mid \pi \text{ es un camino aceptante de } A\}.$$

La aciclicidad se define a partir de los caminos del autómata.

Definición 3.5. Un autómata finito A es **acíclico** si ningún camino de A con al menos una transición empieza y termina en el mismo estado.

La aciclicidad es suficiente para asegurar que el lenguaje aceptado es finito. En los tres lemas siguientes se enuncian la cota de longitud de los caminos, que se usa en el análisis de complejidad, la finitud del lenguaje aceptado y la representación de todo lenguaje finito mediante un autómata acíclico.

Lema 3.1. Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómata finito acíclico. Todo camino de A tiene longitud a lo sumo $|Q| - 1$. En consecuencia, toda cadena aceptada por A tiene longitud a lo sumo $|Q| - 1$ [4].

Lema 3.2. Si A es un autómata finito acíclico, entonces $L(A)$ es finito [4].

Lema 3.3. Todo lenguaje finito puede reconocerse mediante un autómata finito acíclico [4].

En el análisis de complejidad del presente capítulo, el autómata forma parte de la entrada, de modo que el costo se expresa en función de su tamaño. Ese costo depende, ante todo, de la cantidad de caminos aceptantes del autómata. Por ello se introduce la siguiente notación.

Definición 3.6. Sea A un autómata finito acíclico. Se denota por $p(A)$ la cantidad de caminos aceptantes de A .

Como δ asigna a lo sumo un estado a cada par (q, a) , cada cadena aceptada determina un único camino aceptante. En consecuencia,

$$p(A) = |L(A)|.$$

El autómata acíclico es la parte regular de la entrada; en la próxima sección se define el problema del vacío de la intersección.

3.3. Problema de vacío de la intersección

El **problema del vacío de la intersección de una RCG con un lenguaje regular finito** toma como entrada una RCG G y un autómata finito acíclico A , y pregunta si $L(G) \cap L(A) \neq \emptyset$.

La intersección es no vacía si y solo si existe una cadena que se reconoce a la vez por la RCG G y por el autómata A :

$$L(G) \cap L(A) \neq \emptyset \iff \exists w \in \Sigma^* \text{ tal que } w \in L(G) \text{ y } w \in L(A).$$

Decidir ese problema es el objetivo de la sección siguiente.

3.4. Algoritmo de decisión

En esta sección se presenta el algoritmo; su correctitud se demuestra en la subsección 3.4.1 y su complejidad se analiza en la subsección 3.4.2.

Como el autómata de entrada es acíclico, sus caminos aceptantes son finitos. Además, como δ asigna a lo sumo un estado a cada par (q, a) , esos caminos están en correspondencia uno a uno con las cadenas aceptadas. Por tanto, una forma directa de decidir si la intersección es no vacía consiste en recorrer las cadenas aceptadas por el autómata y aplicar, a cada una, un algoritmo de reconocimiento para RCG. El procedimiento es determinista y, en general, no se ejecuta en tiempo polinomial.

Se usará como subrutina un procedimiento de reconocimiento para RCG, que se denota

$$\text{RCG-RECOGNIZE}(G, w).$$

La subrutina devuelve verdadero si y solo si $w \in L(G)$. El reconocimiento de una cadena por una RCG se resuelve en tiempo polinomial en la longitud de la cadena [6, 16].

Antes de enumerar cadenas conviene eliminar los estados que no pueden participar en ningún camino aceptante. Esa reducción evita recorrer ramas que no contribuyen al lenguaje del autómata.

Definición 3.7. Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómata finito. Un estado $q \in Q$ es **útil** si es alcanzable desde q_0 y desde q se puede alcanzar algún estado de aceptación.

El conjunto de estados útiles se obtiene mediante dos recorridos del autómata. En el primero se determinan los estados alcanzables desde q_0 : se parte de $\{q_0\}$ y se agrega $\delta(q, a)$ por cada estado q ya incluido y cada símbolo a para el que $\delta(q, a)$ está definida, hasta que no se agregue ningún estado. En el segundo se determinan los estados desde los que se alcanza un estado de aceptación: se parte de F y se agrega cada estado q tal que $\delta(q, a)$ ya está incluido para algún símbolo a , hasta que no se

agregue ninguno. Un estado es útil cuando pertenece a los dos conjuntos. El autómata $\text{USEFUL-PART}(A)$ tiene por estados los útiles, por estado inicial q_0 , por estados de aceptación los de F que son útiles y por transiciones las $\delta(q, a) = q'$ en las que q y q' son útiles. En cada recorrido, cada estado y cada transición se visitan a lo sumo una vez, de modo que la reducción se calcula en tiempo lineal en el tamaño del autómata.

El pseudocódigo de USEFUL-PART es el siguiente.

$\text{USEFUL-PART}(A)$

```

1   $R = \{q_0\}$ 
2  while there is a transition  $\delta(q, a) = q'$  with  $q \in R$  and  $q' \notin R$ 
3       $R = R \cup \{q'\}$ 
4   $C = F$ 
5  while there is a transition  $\delta(q, a) = q'$  with  $q' \in C$  and  $q \notin C$ 
6       $C = C \cup \{q\}$ 
7   $U = R \cap C$ 
8  return  $(U, \Sigma, \delta', q_0, F \cap U)$ , where  $\delta'$  is the restriction of  $\delta$  to  $U$ 
```

El autómata $\text{USEFUL-PART}(A)$ acepta el mismo lenguaje que A ; cuando no hay estados útiles, ese lenguaje es vacío.

El algoritmo principal se presenta a continuación.

$\text{NONEMPTY-INTERSECTION}(G, A)$

```

1   $A' = \text{USEFUL-PART}(A)$ 
2  if the initial state of  $A'$  does not exist
3      return FALSE
4  return  $\text{SEARCH}(q_0, \varepsilon)$ 
```

La subrutina SEARCH recorre las cadenas aceptadas por el autómata. El parámetro q indica el estado actual y w almacena la cadena que se lee a lo largo del camino desde q_0 hasta ese estado.

$\text{SEARCH}(q, w)$

```

1  if  $q \in F$  and  $\text{RCG-RECOGNIZE}(G, w)$ 
2      return TRUE
3  for each symbol  $a \in \Sigma$  such that  $\delta'(q, a)$  is defined
4       $r = \delta'(q, a)$ 
5      if  $\text{SEARCH}(r, wa)$ 
6          return TRUE
7  return FALSE
```

En el pseudocódigo, δ' denota la función de transición de A' . La línea 1 de SEARCH verifica la pertenencia a la RCG cada vez que el camino recorrido termina en un estado de aceptación. Si la verificación es positiva, la intersección es no vacía y el algoritmo termina. Si la verificación es negativa, el recorrido continúa, ya que desde un estado de aceptación pueden salir transiciones hacia otros caminos aceptantes. La línea 5 actualiza la cadena asociada al camino mediante la concatenación del símbolo leído.

La aciclicidad de A garantiza la terminación de la búsqueda. Los estados $q_0, q_1, \dots, q_d$ que aparecen en una secuencia de llamadas recursivas anidadas forman un camino de A' : en cada paso, de la llamada en el estado q se pasa a la llamada en el estado $\delta'(q, a)$. La profundidad d de la recursión es entonces el número de transiciones de ese camino. Como A' es acíclico, ese camino no contiene dos veces el mismo estado, de modo que ningún estado se repite a lo largo de la recursión. Por el lema 3.1, $d \leq |Q| - 1$. Como en cada llamada se generan a lo sumo $|\Sigma|$ llamadas recursivas, el árbol de recursión es finito y la búsqueda termina.

A continuación se demuestra la correctitud del algoritmo.

3.4.1. Correctitud

La demostración separa tres hechos. El primero establece que toda cadena que se evalúa con RCG-RECOGNIZE pertenece al lenguaje del autómata. El segundo establece que toda cadena aceptada por el autómata se evalúa exactamente una vez. El tercero se apoya en la correctitud de RCG-RECOGNIZE.

Lema 3.4. *Toda cadena w para la que el algoritmo ejecuta RCG-RECOGNIZE(G, w) pertenece a $L(A)$.*

Demostración. La subrutina SEARCH ejecuta RCG-RECOGNIZE(G, w) solo en la línea 1, cuando el estado actual q pertenece a F . El valor de w se construye concatenando los símbolos que se leen desde q_0 hasta q . Por tanto, existe un camino desde q_0 hasta un estado de aceptación a lo largo del cual se lee w . Por la definición de aceptación, $w \in L(A')$. Como $A' = \text{USEFUL-PART}(A)$ acepta el mismo lenguaje que A , se tiene $L(A') = L(A)$. Luego $w \in L(A)$. $\square$

Lema 3.5. *Para toda cadena $w \in L(A)$, el algoritmo ejecuta RCG-RECOGNIZE(G, w) exactamente una vez.*

Demostración. Sea $w = a_1 a_2 \dots a_m$ una cadena de $L(A)$. Como para cada par (q, a) hay a lo sumo un estado $\delta(q, a)$, existe un único camino de A que comienza en q_0 y a lo largo del cual se lee w :

$$q_0 a_1 q_1 a_2 q_2 \dots a_m q_m.$$

Además, como $w \in L(A)$, se tiene $q_m \in F$. Todos los estados de ese camino son útiles: cada uno es alcanzable desde q_0 por el prefijo correspondiente, y desde cada uno se

alcanza un estado de aceptación por el sufijo correspondiente. Por tanto, el camino también pertenece a A' .

La subrutina SEARCH recorre, para cada estado actual, todos los símbolos a para los cuales $\delta'(q, a)$ está definida. Por inducción sobre la longitud de los prefijos de w , el algoritmo realiza una llamada recursiva asociada al único prefijo de longitud i de w , para cada $0 \leq i \leq m$. Al llegar al estado q_m , la cadena acumulada es w y se cumple $q_m \in F$. En ese momento se ejecuta RCG-RECOGNIZE(G, w).

La ejecución es única porque en A' hay a lo sumo un estado $\delta'(q, a)$ por cada par (q, a) : ninguna cadena se lee a lo largo de dos caminos distintos desde q_0 . Por tanto, el algoritmo no puede alcanzar un estado de aceptación con la misma cadena w por dos recorridos diferentes. $\square$

Lema 3.6. *Para toda cadena w , la subrutina RCG-RECOGNIZE(G, w) devuelve verdadero si y solo si $w \in L(G)$.*

Demostración. El reconocimiento de una cadena por una RCG se decide en tiempo polinomial [6, 16]. RCG-RECOGNIZE es un procedimiento que resuelve ese problema y devuelve verdadero si y solo si $w \in L(G)$. $\square$

Teorema 3.1. *El algoritmo NONEMPTY-INTERSECTION(G, A) devuelve verdadero si y solo si*

$$L(G) \cap L(A) \neq \emptyset.$$

Demostración. Supóngase primero que el algoritmo devuelve verdadero. Entonces, para alguna cadena w , se ejecutó RCG-RECOGNIZE(G, w) y la subrutina devolvió verdadero. Por el lema 3.4, $w \in L(A)$. Por el lema 3.6, $w \in L(G)$. Luego $w \in L(G) \cap L(A)$, y la intersección es no vacía.

Supóngase ahora que $L(G) \cap L(A) \neq \emptyset$. Entonces existe una cadena w tal que $w \in L(G)$ y $w \in L(A)$. Por el lema 3.5, el algoritmo ejecuta RCG-RECOGNIZE(G, w) exactamente una vez. Por el lema 3.6, esa ejecución devuelve verdadero. Por tanto, el algoritmo devuelve verdadero. $\square$

A continuación se analiza la complejidad del algoritmo.

3.4.2. Complejidad

El análisis depende de dos componentes. El primero es la enumeración de cadenas aceptadas por el autómata acíclico. El segundo es el reconocimiento de cada cadena mediante la RCG. En la presente subsección se expresan ambos costos de manera separada.

Sea $|A| = |Q| + |\delta|$ el tamaño del autómata, donde $|\delta|$ denota la cantidad de pares (q, a) para los cuales la transición está definida. Sea $p(A)$ el número de caminos aceptantes de A . Como se estableció en la sección 3.2, $p(A) = |L(A)|$.

Después de eliminar estados no útiles, todo camino recorrido por el algoritmo es prefijo de algún camino aceptante. Como cada camino aceptante tiene longitud a lo sumo $|Q| - 1$, el número de prefijos recorridos está acotado por

$$p(A)|Q|.$$

Por tanto, el costo de la enumeración, sin contar las llamadas a RCG-RECOGNIZE, queda acotado por

$$O(p(A)|Q|).$$

Sea $R_G(n)$ el tiempo que demora RCG-RECOGNIZE(G, w) sobre una cadena w de longitud n . Para una RCG fija, $R_G(n)$ es polinomial en n . Si la gramática forma parte de la entrada, el costo es polinomial en la longitud de la cadena para cada gramática fijada, y el exponente depende de parámetros estructurales de la gramática, como la cantidad de variables que pueden recibir rangos en una cláusula [6, 16].

Por el lema 3.1, toda cadena aceptada tiene longitud a lo sumo $|Q| - 1$. Luego el costo total del algoritmo se acota por

$$O(|A| + p(A)|Q| + p(A)R_G(|Q| - 1)).$$

De forma equivalente,

$$O(|A| + p(A)(|Q| + R_G(|Q| - 1))).$$

La cota anterior es polinomial si $p(A)$ está acotado por un polinomio en $|A|$. Sin embargo, $p(A)$ no está polinomialmente acotado para todos los autómatas acíclicos. En la sección 3.1 ya se mostró un autómata así: el que reconoce $L_n = \{0, 1\}^n$, con $n + 1$ estados, acíclico y con 2^n cadenas aceptadas. Por la igualdad $p(A) = |L(A)|$, ese autómata tiene 2^n caminos aceptantes con un tamaño lineal en n .

Por tanto, la enumeración determinista decide el problema, pero su costo puede ser exponencial en el tamaño del autómata, incluso para autómatas acíclicos.

El costo del algoritmo depende de cuántas cadenas acepta la representación regular. Si una clase de autómatas acíclicos tiene polinomialmente muchas cadenas aceptadas, el vacío de la intersección se decide en tiempo polinomial para cada RCG fija; si admite autómatas con exponencialmente muchas cadenas, ese costo es exponencial. Esa condición es trivial: enumerar es barato cuando hay poco que enumerar, de modo que la enumeración no es una vía factible para decidir el vacío de la intersección, pero sí necesaria para entender el problema y por qué conviene resolverlo sobre un único objeto que exprese lo que reconocen ambos formalismos.

Generar cada cadena aceptada en tiempo polinomial no es lo mismo que enumerarlas todas en tiempo polinomial. La cantidad de cadenas aceptadas puede ser exponencial, de modo que recorrerlas todas cuesta tiempo exponencial aun cuando

cada una se obtenga en tiempo polinomial. Para decidir el vacío de la intersección hace falta, en el peor caso, recorrer todas: cuando la intersección es vacía, ninguna cadena aceptada pertenece a $L(G)$, y el recorrido solo concluye al agotarlas.

En este capítulo se definió el problema del vacío de la intersección de una RCG con un lenguaje regular finito y se dio un algoritmo de fuerza bruta que lo decide cuando el lenguaje regular se representa con un autómata acíclico; se demostró su correctitud y se caracterizó su costo, exponencial en el peor caso. En el próximo capítulo se modela el SAT como ese vacío, asociando a cada fórmula booleana un autómata booleano y una RCG de consistencia.

Capítulo 4

El SAT como vacío de la intersección de una RCG con un lenguaje regular finito

Con la no linealidad de las RCG se expresa la consistencia de cualquier fórmula, sin la restricción sobre el orden de las ocurrencias presente en SATCF y SATSRC. Esa modelación general es el punto de partida para buscar restricciones no lineales con vacío tratable, y con ellas fragmentos del SAT polinomiales más amplios que el de SATSRC.

En la sección 4.1 se construye esa modelación y se demuestra la reducción desde el SAT; en la sección 4.2 se lee el resultado en términos del SAT y se desarrollan sus consecuencias.

4.1. Modelación del SAT con RCG

A una fórmula con n ocurrencias de variable booleana se le asocian el autómata booleano del capítulo 2, finito y acíclico, y una RCG no lineal¹ que impone la consistencia entre las ocurrencias de una misma variable booleana.

Sea $\ell_1 \cdots \ell_n$ la secuencia de ocurrencias de la fórmula, con ℓ_i la variable booleana de la ocurrencia i . Por ejemplo, la fórmula $x_1 \wedge (x_2 \vee x_1)$ tiene la secuencia de ocurrencias $x_1 x_2 x_1$, con $n = 3$, de modo que $\ell_1 = x_1$, $\ell_2 = x_2$ y $\ell_3 = x_1$. Las ocurrencias 1 y 3 son de la misma variable booleana x_1 .

La gramática de consistencia G_{cons} tiene por terminales $T = \{0, 1\}$, por variables $X, W_0, W_1, \dots, W_n$ y tres juegos de predicados, todos de aridad uno: el predicado inicial

¹La consistencia se puede modelar con otras RCG no lineales; la que se usa aquí es solo una propuesta.

S , un predicado Eq_v por cada variable booleana v y los predicados $L_0, L_1, \dots, L_n$. El predicado L_j reconoce las cadenas de longitud j , mediante las cláusulas

$$L_j(cX) \rightarrow L_{j-1}(X) \quad (c \in \{0, 1\}, 1 \leq j \leq n), \quad L_0(\varepsilon) \rightarrow \varepsilon.$$

Por ejemplo, L_3 reconoce la cadena 101 mediante la derivación $L_3(101) \rightarrow L_2(01) \rightarrow L_1(1) \rightarrow L_0(\varepsilon) \rightarrow \varepsilon$, que consume un símbolo en cada paso hasta llegar a la cadena vacía.

Para cada variable booleana v , con ocurrencias en las posiciones $i_1 < i_2 < \dots < i_r$, el predicado Eq_v reconoce las cadenas de longitud n en las que esas posiciones llevan el mismo símbolo. Sus cláusulas son dos, una por cada símbolo $b \in \{0, 1\}$:

$$Eq_v(W_0 b W_1 b \dots b W_r) \rightarrow L_{g_0}(W_0) L_{g_1}(W_1) \dots L_{g_r}(W_r).$$

Es decir, para cada variable booleana v las dos cláusulas son

$$\begin{aligned} Eq_v(W_0 0 W_1 0 \dots 0 W_r) &\rightarrow L_{g_0}(W_0) L_{g_1}(W_1) \dots L_{g_r}(W_r), \\ Eq_v(W_0 1 W_1 1 \dots 1 W_r) &\rightarrow L_{g_0}(W_0) L_{g_1}(W_1) \dots L_{g_r}(W_r). \end{aligned}$$

En el argumento del lado izquierdo, el símbolo b ocupa las posiciones de v y cada variable W_k cubre una subcadena: la anterior a la primera ocurrencia, la comprendida entre dos ocurrencias consecutivas o la posterior a la última. Esas subcadenas tienen longitudes $g_0 = i_1 - 1$, $g_k = i_{k+1} - i_k - 1$ y $g_r = n - i_r$, y cada una se reconoce por el predicado L_{g_k} correspondiente. Una vez fijada la fórmula, las posiciones i_k y las longitudes g_k son constantes, de modo que el conjunto de cláusulas es finito y cada cláusula tiene la forma de la definición 2.2. La figura 4.1 ilustra esa partición para una variable booleana con dos ocurrencias.

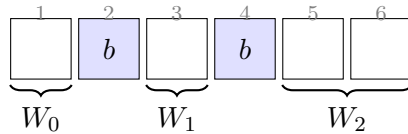


Figura 4.1: Partición del argumento de Eq_v para una variable booleana v con ocurrencias en las posiciones $i_1 = 2$ e $i_2 = 4$ de una cadena de longitud $n = 6$. Las celdas sombreadas llevan el símbolo b en las posiciones de v ; cada W_k cubre la subcadena entre ocurrencias consecutivas, con longitudes $g_0 = 1$, $g_1 = 1$ y $g_2 = 2$.

La consistencia de la fórmula es la conjunción² de esas restricciones, y se expresa con una cláusula no lineal con el mismo argumento en los m predicados Eq_v , donde m es la cantidad de variables booleanas de la fórmula:

$$S(X) \rightarrow Eq_{v_1}(X) Eq_{v_2}(X) \dots Eq_{v_m}(X).$$

²Por conjunción se entiende que todas las restricciones deben cumplirse simultáneamente.

La variable X es la misma en los m predicados, así que las m restricciones se aplican a la misma cadena. Con esta cláusula queda completo el conjunto de cláusulas y, con él, la gramática G_{cons} . El lenguaje reconocido por G_{cons} se denota $L(G_{\text{cons}})$; el lema siguiente lo caracteriza.

Lema 4.1. *Sea G_{cons} la gramática de consistencia de una fórmula booleana con n ocurrencias de variable. El lenguaje $L(G_{\text{cons}})$ es el conjunto de las cadenas consistentes de la fórmula.*

Demostración. Primero se establece, por inducción sobre j , que $L_j(u)$ deriva en la cadena vacía si y solo si $|u| = j$.

En el caso base, $j = 0$, la única cláusula con L_0 en el lado izquierdo es $L_0(\varepsilon) \rightarrow \varepsilon$, que solo aplica cuando el argumento es la cadena vacía; así, $L_0(u)$ deriva en la cadena vacía si y solo si $u = \varepsilon$, esto es, $|u| = 0$.

En el paso inductivo, $j \geq 1$, se supone como hipótesis que $L_{j-1}(u')$ deriva en la cadena vacía si y solo si $|u'| = j - 1$. Las únicas cláusulas con L_j en el lado izquierdo son $L_j(cX) \rightarrow L_{j-1}(X)$, con $c \in \{0, 1\}$, de modo que $L_j(u)$ deriva en la cadena vacía solo si $u = cu'$ para algún símbolo c , y la derivación continúa con $L_{j-1}(u')$. Entonces $L_j(u)$ deriva en la cadena vacía si y solo si $L_{j-1}(u')$ lo hace, lo que por la hipótesis equivale a $|u'| = j - 1$; como $|u| = |u'| + 1$, esto equivale a $|u| = j$.

Sea w una cadena consistente; como lleva un símbolo por ocurrencia, $|w| = n$. Para cada variable booleana v , sea b el símbolo común de sus posiciones en w . Entonces w se descompone como $u_0 b u_1 b \cdots b u_r$ con $|u_k| = g_k$: en el patrón del símbolo b , la sustitución de rango asocia cada variable W_k a la subcadena u_k , y cada predicado $L_{g_k}(u_k)$ deriva en la cadena vacía. Luego $Eq_v(w)$ deriva en la cadena vacía. Como esto vale para las m variables booleanas, con la cláusula de S se deriva en la cadena vacía y $w \in L(G_{\text{cons}})$.

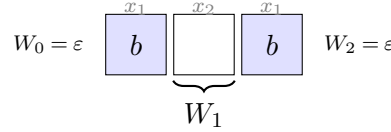
Sea ahora $w \in L(G_{\text{cons}})$. La única cláusula de S es $S(X) \rightarrow Eq_{v_1}(X) \cdots Eq_{v_m}(X)$, de modo que cada $Eq_v(w)$ deriva en la cadena vacía. En esa derivación se aplica una de las dos cláusulas de Eq_v , con algún símbolo b : entonces $w = u_0 b u_1 b \cdots b u_r$, y cada predicado $L_{g_k}(u_k)$ deriva en la cadena vacía, así que $|u_k| = g_k$. Como $g_0 + g_1 + \cdots + g_r + r = n$ por la definición de las longitudes g_k , se tiene $|w| = n$, y las posiciones $i_1, \dots, i_r$ de w llevan el símbolo b . Como esto vale para cada variable booleana, w es consistente. $\square$

Por ejemplo, considere la fórmula $x_1 \wedge (x_2 \vee x_1)$ del capítulo 2, cuya secuencia de ocurrencias es $x_1 x_2 x_1$, con $n = 3$ y $m = 2$. La variable booleana x_1 ocurre en las posiciones 1 y 3, con subcadenas de longitudes $g_0 = 0$, $g_1 = 1$ y $g_2 = 0$; la variable booleana x_2 ocurre en la posición 2, con subcadenas de longitudes $g_0 = 1$ y $g_1 = 1$. Las cláusulas de G_{cons} son las siguientes, omitidas las de L_2 y L_3 , que en ningún patrón

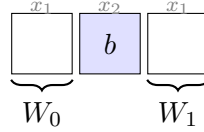
aparecen:

$$\begin{aligned}
 S(X) &\rightarrow Eq_{x_1}(X) Eq_{x_2}(X), \\
 Eq_{x_1}(W_0 0 W_1 0 W_2) &\rightarrow L_0(W_0) L_1(W_1) L_0(W_2), \\
 Eq_{x_1}(W_0 1 W_1 1 W_2) &\rightarrow L_0(W_0) L_1(W_1) L_0(W_2), \\
 Eq_{x_2}(W_0 0 W_1) &\rightarrow L_1(W_0) L_1(W_1), \\
 Eq_{x_2}(W_0 1 W_1) &\rightarrow L_1(W_0) L_1(W_1), \\
 L_1(0X) &\rightarrow L_0(X), \quad L_1(1X) \rightarrow L_0(X), \quad L_0(\varepsilon) \rightarrow \varepsilon.
 \end{aligned}$$

La figura 4.2 muestra, según la secuencia de ocurrencias $x_1 x_2 x_1$, qué posiciones cubre cada W_k en los patrones de Eq_{x_1} y Eq_{x_2} .



(a) patrón de Eq_{x_1} : las posiciones 1 y 3 llevan b



(b) patrón de Eq_{x_2} : la posición 2 lleva b

Figura 4.2: Subcadenas que cubre cada W_k sobre la secuencia de ocurrencias $x_1 x_2 x_1$. La fila superior indica a qué variable booleana pertenece cada posición. En (a), para Eq_{x_1} , las posiciones 1 y 3 son de x_1 y W_1 cubre la posición 2, con W_0 y W_2 vacías ($g_0 = g_2 = 0$, $g_1 = 1$). En (b), para Eq_{x_2} , la posición 2 es de x_2 y W_0, W_1 cubren las posiciones 1 y 3 ($g_0 = g_1 = 1$).

La cadena 101 es consistente y se reconoce por G_{cons} mediante las derivaciones siguientes:

$$\begin{aligned}
 S(101) &\rightarrow Eq_{x_1}(101) Eq_{x_2}(101), \\
 Eq_{x_1}(101) &\rightarrow L_0(\varepsilon) L_1(0) L_0(\varepsilon), \\
 Eq_{x_2}(101) &\rightarrow L_1(1) L_1(1), \\
 L_1(0) &\rightarrow L_0(\varepsilon), \\
 L_1(1) &\rightarrow L_0(\varepsilon), \\
 L_0(\varepsilon) &\rightarrow \varepsilon.
 \end{aligned}$$

A continuación se describen los pasos.

- **Primer paso:** en la cláusula $S(X) \rightarrow Eq_{x_1}(X) Eq_{x_2}(X)$, la sustitución de rango asocia la variable X al valor $X = 101$. De esta forma se deriva en $Eq_{x_1}(101) Eq_{x_2}(101)$.

- **Segundo paso:** en el patrón del símbolo 1 de Eq_{x_1} , la sustitución de rango asocia las variables W_0, W_1, W_2 a los valores $W_0 = \varepsilon, W_1 = 0$ y $W_2 = \varepsilon$. Con estos valores se deriva en $L_0(\varepsilon) L_1(0) L_0(\varepsilon)$.
- **Tercer paso:** en el patrón del símbolo 0 de Eq_{x_2} , la sustitución de rango asocia las variables W_0, W_1 a los valores $W_0 = 1$ y $W_1 = 1$. Con estos valores se deriva en $L_1(1) L_1(1)$.
- **Cuarto paso:** cada predicado L_1 deriva en $L_0(\varepsilon)$, y cada predicado $L_0(\varepsilon)$ deriva en la cadena vacía por la cláusula terminal, con lo que la cadena 101 queda reconocida y $101 \in L(G_{\text{cons}})$.

La figura 4.3 muestra el árbol de derivación completo de 101 en G_{cons} .

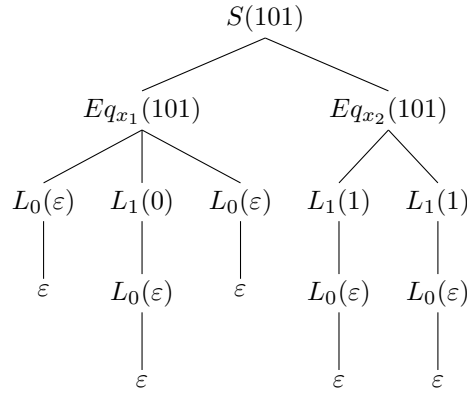


Figura 4.3: Árbol de derivación de la cadena 101 en G_{cons} . Los hijos de cada nodo son los predicados del lado derecho de la cláusula que se le aplica; las hojas ε corresponden a la cláusula terminal $L_0(\varepsilon) \rightarrow \varepsilon$.

La cadena 110, con la que en el capítulo 2 se llega al estado V del autómata booleano, no es consistente. En $Eq_{x_1}(110)$ ningún patrón admite una sustitución de rango: el del símbolo 0 exige 0 en la posición 1, donde la cadena lleva 1, y el del símbolo 1 exige 1 en la posición 3, donde la cadena lleva 0. Como $Eq_{x_1}(110)$ no deriva en la cadena vacía, $110 \notin L(G_{\text{cons}})$. La figura 4.4 lo ilustra.

La correctitud de esta modelación se demuestra en el teorema siguiente.

Teorema 4.1. Sean G_{cons} la gramática de consistencia de una fórmula booleana y A su autómata booleano. La fórmula es satisfacible si y solo si $L(G_{\text{cons}}) \cap L(A) \neq \emptyset$.

Demostración. El lenguaje del autómata booleano está formado por las cadenas para las que la fórmula es verdadera cuando cada ocurrencia se trata por separado [10].

A cada interpretación I de la fórmula le corresponde la cadena $w_I = b_1 b_2 \dots b_n$, de longitud n , el número de ocurrencias de variable booleana de la fórmula. Cada b_i

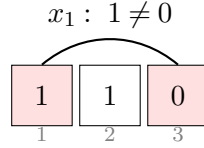


Figura 4.4: La cadena 110 no es consistente: las dos ocurrencias de x_1 (posiciones 1 y 3, sombreadas) llevan símbolos distintos.

es 1 si ℓ_i , la variable booleana de la ocurrencia i , es verdadera en I , y 0 si es falsa. La cadena w_I es consistente: cada variable booleana tiene un único valor en I , así que todas sus posiciones en w_I llevan ese mismo valor. Recíprocamente, toda cadena consistente es w_I para la interpretación I en la que cada variable booleana toma el símbolo común de sus posiciones.

Supóngase que la fórmula es satisfacible. Entonces la fórmula es verdadera en alguna interpretación I . La cadena w_I es consistente, de modo que $w_I \in L(G_{\text{cons}})$ por el lema 4.1, y $w_I \in L(A)$. Luego la intersección es no vacía.

Supóngase ahora que la intersección es no vacía. Entonces alguna cadena w pertenece a $L(G_{\text{cons}})$ y a $L(A)$. Por el lema 4.1, w es consistente, de modo que $w = w_I$ para la interpretación I en la que cada variable booleana toma el símbolo común de sus posiciones. De $w_I \in L(A)$ se sigue que la fórmula es verdadera en I . Por tanto, la fórmula es satisfacible. $\square$

El SAT se reduce entonces al problema de vacío de la intersección que se resuelve en el capítulo 3. A continuación se lee esta reducción en términos del SAT y se desarrollan sus consecuencias.

4.2. Consecuencias para el SAT y subclases tratables

En la enumeración del capítulo 3 se recorren las cadenas aceptadas por el autómata booleano y se aplica a cada una el reconocimiento de la RCG de consistencia; las dos partes de la instancia tienen costo polinomial. El autómata booleano tiene $O(n^2)$ estados, con n la cantidad de ocurrencias de variables booleanas en la fórmula [10]. La gramática G_{cons} tiene tamaño $O(m+n)$, con m la cantidad de variables booleanas distintas: dos cláusulas por variable booleana, más las $O(n)$ cláusulas de los predicados L_j . Como $m \leq n$, ese tamaño es $O(n)$. El reconocimiento con G_{cons} es polinomial: para cada cadena hay a lo sumo una sustitución de rango por cláusula, porque los rangos de las variables W_k se determinan a partir de las longitudes g_k , y esa sustitución se comprueba en tiempo lineal; el reconocimiento de una cadena de longitud n cuesta

entonces $R_{G_{\text{cons}}}(n) = O(nm)$.

Toda cadena aceptada por el autómata booleano tiene longitud n , de modo que cada llamada a RCG-RECOGNIZE cuesta $O(nm)$. Con $|Q| = O(n^2)$, la cota de la subsección 3.4.2 queda en

$$O(n^2 + p(A)(n^2 + nm)) = O(n^2 + p(A)n^2),$$

pues $m \leq n$. El factor dominante es $p(A)$, la cantidad de cadenas aceptadas por el autómata booleano, que para una fórmula con muchas interpretaciones verdaderas bajo el supuesto de independencia es del orden de 2^n . En el peor caso, la cota queda en

$$O(n^2 + 2^n n^2) = O(2^n n^2).$$

La dificultad no está entonces en el tamaño de la instancia ni en el reconocimiento, sino en el vacío de la intersección: decidir si $L(G_{\text{cons}}) \cap L(A)$ es vacío es el SAT, y para una RCG con un autómata finito acíclico ese problema es NP-completo [4].

De esta modelación se siguen tres consecuencias para el SAT. Con ella y la enumeración del capítulo 3 se decide la satisfacibilidad de cualquier fórmula booleana, sin la restricción de orden de SATCF y SATSRC. El alcance es general, pero la decisión es por fuerza bruta: el costo es exponencial, y solo es polinomial cuando el autómata booleano tiene una cantidad de caminos aceptantes acotada por un polinomio en n ; aquí no se caracterizan las fórmulas booleanas que cumplen esa condición. La gramática de consistencia tiene aridez uno: con la no linealidad, la aridez de las cláusulas no crece con los cruces. En SATSRC el vacío de la intersección se decide en tiempo $O(n^k)$, con k la aridez: el exponente crece con la aridez necesaria para los cruces, y la cota deja de ser polinomial cuando esa aridez no está acotada. Con la no linealidad la aridez se mantiene en uno, a costa de que el vacío de la intersección sea NP-completo. Por último, la modelación es una propuesta concreta de cómo expresar el SAT con una RCG. Con un procedimiento polinomial para el vacío de la intersección de una RCG con un autómata acíclico se decidiría el SAT en tiempo polinomial, esto es, $P = NP$. Con una subclase no lineal cuyo vacío de la intersección sea polinomial se obtendría, en cambio, un fragmento tratable; pero esta modelación se beneficiaría de esa tratabilidad solo si su gramática de consistencia pertenece a la subclase, y, si no, sirve únicamente como guía para adaptar la construcción a una subclase de ese tipo, sin garantía de que sea posible.

En este capítulo se modeló el SAT como el vacío de la intersección de una RCG con un lenguaje regular finito: a cada fórmula booleana se le asociaron un autómata booleano y una RCG de consistencia no lineal, se demostró la reducción desde el SAT y se leyeron sus consecuencias. Para estudiar ese vacío hace falta una representación simbólica de la intersección. En el próximo capítulo se construye una: la gramática producto entre una RCG y un lenguaje regular; con ella el problema general no se

vuelve polinomial, pero se obtiene el objeto sobre el que estudiar ese vacío: buscar subclases no lineales con vacío polinomial, un algoritmo polinomial para el caso general o argumentos de que no existe.

Capítulo 5

Construcción de la gramática producto de una gramática de concatenación de rango y un lenguaje regular finito

La literatura provee construcciones de la intersección con un lenguaje regular para las gramáticas libres del contexto y las de concatenación de rango simple [3, 4]. En las gramáticas de concatenación de rango, una misma variable puede aparecer más de una vez en el lado derecho de una cláusula. Esta no linealidad introduce una dificultad que las construcciones anteriores no contemplan. Este capítulo aporta una construcción que, a partir de una gramática de concatenación de rango G y un autómata finito acíclico A , produce una nueva gramática de concatenación de rango G' , la **gramática producto**¹, cuyo lenguaje es exactamente $L(G) \cap L(A)$.

La construcción se apoya en una noción de rango adaptada al autómata, que se introduce en la sección siguiente.

5.1. El rango sobre un autómata

En una RCG el reconocimiento se hace por rangos; en un autómata, por estados. Para reconocer una cadena con ambos hay que coordinarlos: de cada rango se necesita saber en qué estado queda el autómata al empezar y al terminar de leer su subcadena. Pero el rango (i, j) es un par de posiciones en una cadena fija, ajeno a los estados del autómata.

¹El término se usa por analogía con el autómata producto, la construcción estándar para la intersección de dos lenguajes regulares [14].

Hace falta, entonces, una noción de rango ligada a los estados del autómata. Para ello, cada subcadena se caracteriza por el par de estados entre los que transita el autómata al leerla. Para fijar ese par hace falta saber en qué estado termina el autómata al comenzar en un estado y leer una cadena, lo que da lugar a la siguiente definición.

Definición 5.1. Sea $A = (Q, \Sigma, \delta, q_0, F)$ un autómata finito determinista. La función de transición δ se extiende a cadenas mediante la **función de transición extendida** $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, que se define por

$$\hat{\delta}(q, \varepsilon) = q \quad \text{y} \quad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

para $w \in \Sigma^*$ y $a \in \Sigma$ [14]. Así, $\hat{\delta}(q, u)$ es el estado en el que el autómata termina al comenzar en el estado q y leer la cadena u .

En los ejemplos de este capítulo se retoma la modelación del capítulo 4 de la fórmula $x_1 \wedge (x_2 \vee x_1)$, cuya secuencia de ocurrencias es $x_1 x_2 x_1$. Su autómata booleano A se muestra en la figura 2.6. Es finito y acíclico: reconoce las cadenas de longitud tres que hacen verdadera la fórmula cuando cada ocurrencia se trata por separado. Su estado inicial es q_0 , su único estado de aceptación es V , y su lenguaje es $L(A) = \{101, 110, 111\}$.

Por ejemplo, al leer 101 sobre A se extiende $\hat{\delta}$ símbolo a símbolo,

$$\hat{\delta}(q_0, 1) = q_1, \quad \hat{\delta}(q_0, 10) = q_2 \quad \text{y} \quad \hat{\delta}(q_0, 101) = V,$$

un estado de aceptación; para la cadena 100, en cambio, se obtiene $\hat{\delta}(q_0, 100) = F$. La figura 5.1 muestra estos cuatro caminos en A .

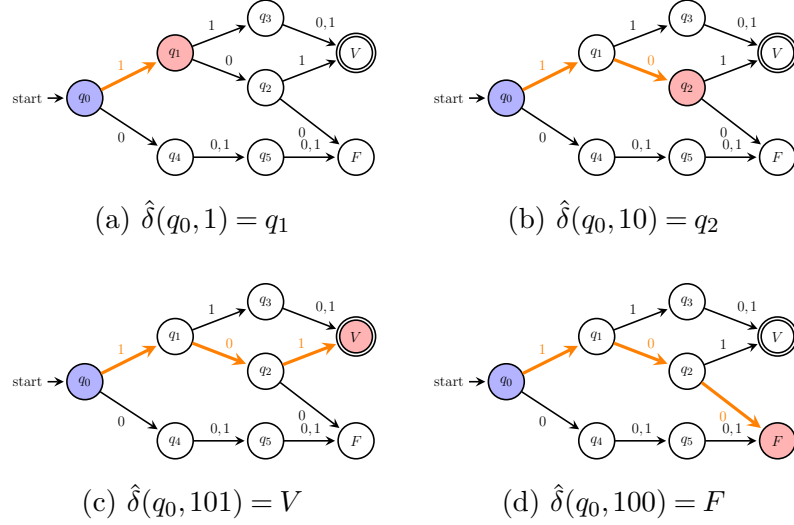


Figura 5.1: Caminos en A de los cuatro ejemplos de $\hat{\delta}$. En cada uno, el estado de partida q_0 va en azul y el de llegada en rojo; las transiciones resaltadas forman el camino, a lo largo del cual se lee la cadena correspondiente.

La función $\hat{\delta}$ determina, para cada cadena y cada estado de partida, el estado en que el autómata termina, lo que permite definir el rango sobre el autómata como un par de estados y precisar cuándo una cadena lo realiza.

Definición 5.2. Un **rango sobre A** es un par de estados $(q, q') \in Q \times Q$. Una cadena u **realiza** el rango (q, q') si $\hat{\delta}(q, u) = q'$.

Un rango sobre A describe a la vez todas las cadenas que lo realizan. Cuando a una variable se le asocia el rango (q, q') , la gramática debe derivar la subcadena correspondiente, que a la vez debe realizar ese rango.

En el ejemplo anterior, al leer 101 se recorre en A el camino $q_0 1 q_1 0 q_2 1 V^2$. Así, el primer símbolo realiza (q_0, q_1) ; el segundo, (q_1, q_2) ; el tercero, (q_2, V) ; la subcadena 10 realiza (q_0, q_2) ; y la cadena entera 101 realiza (q_0, V) , que parte del estado inicial y termina en el de aceptación.

En la sección siguiente, las nociones planteadas para las cadenas se generalizan al caso de los argumentos.

5.2. El rango de un argumento sobre un autómata

La sección anterior estableció cuándo una cadena realiza un rango sobre A . Un argumento de una cláusula es una secuencia de terminales y variables, y las variables

²Recuérdese que en la notación de camino del capítulo 3 estados y símbolos se alternan.

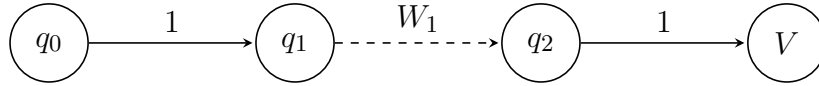
no son símbolos que el autómata pueda leer. Hace falta entonces decir qué rango sobre A le corresponde a un argumento de esa forma.

Definición 5.3. Sea $\alpha = \alpha_1 \cdots \alpha_n$ un argumento, donde cada $\alpha_i \in T \cup V$. Una **asignación de rangos sobre A** para α es una función ρ que asigna a cada α_i un rango sobre A , $\rho(\alpha_i) = (q_i, q'_i)$, con dos condiciones:

1. $q'_i = q_{i+1}$ para $1 \leq i < n$;
2. si α_i es un terminal a , entonces $\delta(q_i, a) = q'_i$.

Cuando existe una asignación así, el argumento α realiza el rango sobre A (q_1, q'_n) .

Por ejemplo, considérese el argumento $W_0 1 W_1 1 W_2$ del lado izquierdo de la cláusula $Eq_{x_1}(W_0 1 W_1 1 W_2) \rightarrow L_0(W_0) L_1(W_1) L_0(W_2)$ de G_{cons} , que reconoce las cadenas de longitud tres con 1 en las dos posiciones de x_1 . Sobre la cadena consistente 101, con $W_0 = \varepsilon$, $W_1 = 0$ y $W_2 = \varepsilon$, la asignación de rangos sobre A es la siguiente, con W_0 y W_2 vacíos en los extremos:



Esta asignación cumple las dos condiciones de la definición de asignación de rangos sobre A . La primera, que el final de cada rango sea el inicio del siguiente, se ve al recorrer los rangos: W_0 realiza (q_0, q_0) ; el primer 1, (q_0, q_1) ; W_1 , (q_1, q_2) ; el segundo 1, (q_2, V) ; y W_2 , (V, V) . La segunda, que cada terminal a con rango (q_i, q'_i) cumpla $\delta(q_i, a) = q'_i$, la satisfacen los dos terminales 1: $\delta(q_0, 1) = q_1$ y $\delta(q_2, 1) = V$. El argumento realiza entonces (q_0, V) , que parte de q_0 y termina en el estado de aceptación V .

Con la cláusula del símbolo 0, $Eq_{x_1}(W_0 0 W_1 0 W_2)$, se da el caso contrario. Al leer la cadena consistente 000 se recorre el camino $q_0 0 q_4 0 q_5 0 F$, de modo que el argumento realiza (q_0, F) , que no termina en el estado de aceptación: 000 es consistente, pero hace falsa la fórmula.

La sección siguiente presenta la construcción de la gramática producto.

5.3. Construcción de la gramática producto

En esta sección se desarrolla la construcción de la gramática producto. Primero se presenta el algoritmo. A continuación se impone una restricción sobre la gramática de entrada. Después se establece una condición para que las ocurrencias de una misma variable reciban el mismo rango. Luego se da el pseudocódigo. Por último se demuestra su correctitud y se analizan su tamaño y la complejidad de construirla.

5.3.1. Algoritmo

El objetivo es producir una gramática $G' = (N', T, V, P', S')$ que reconozca las cadenas que G reconoce y A acepta. Una cadena w pertenece a $L(A)$ si el rango sobre A que w realiza parte del estado inicial y termina en un estado final, así que G' no solo debe reconocer las cadenas de $L(G)$, sino reflejar también qué rango sobre A realiza cada argumento de cada predicado durante la derivación. Para reflejarlo, los predicados de G' se diferencian según el rango sobre A que realiza cada uno de sus argumentos: cada predicado de G da lugar a un predicado de G' por cada vector de rangos sobre A .

Definición 5.4. Sean B un predicado de G de aridad k y $\mathbf{q} = ((q_1, q'_1), \dots, (q_k, q'_k))$ un vector de k rangos sobre A , uno por cada argumento de B . El par $(B, \mathbf{q})$ se llama **predicado indexado** y se denota $B[\mathbf{q}]$.

Por ejemplo, el autómata A tiene ocho estados, así que cada predicado de G_{cons} da lugar a hasta sesenta y cuatro predicados indexados, uno por cada par de estados. En el reconocimiento de 101 aparecen $S[(q_0, V)]$, $Eq_{x_1}[(q_0, V)]$ y $Eq_{x_2}[(q_0, V)]$, junto con predicados indexados de L_0 y L_1 .

El predicado inicial de G' es un predicado S' de aridad 1. El conjunto de predicados N' consta de los predicados indexados y del predicado inicial S' .

Las cláusulas de G' se generan a partir de las cláusulas de G y de las asignaciones de rangos sobre A a sus ocurrencias de símbolos. Para cada cláusula

$$B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$$

de G y cada asignación que cumpla las condiciones de la sección 5.2, indexando cada predicado B_j por el vector $\mathbf{q}_j$ de los rangos de sus argumentos se obtiene una **cláusula indexada**³:

$$B_0[\mathbf{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1[\mathbf{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\mathbf{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m}).$$

Por ejemplo, en la cláusula $L_1(1X) \rightarrow L_0(X)$ de G_{cons} , con el terminal 1 en el rango (q_0, q_1) , pues $\delta(q_0, 1) = q_1$, y la variable X en (q_1, q_1) , el argumento $1X$ realiza (q_0, q_1) . La cláusula indexada que se obtiene es

$$L_1[(q_0, q_1)](1X) \rightarrow L_0[(q_1, q_1)](X).$$

La construcción genera una cláusula indexada por cada asignación de rangos que cumpla las condiciones, de modo que cada cláusula de G_{cons} da lugar a muchas.

³Una cláusula indexada es una cláusula de la gramática producto G' , formada por predicados indexados.

Cuando el lado derecho de una cláusula de G es vacío, solo se indexa el lado izquierdo: la cláusula $B(\alpha_1, \dots, \alpha_k) \rightarrow \varepsilon$ produce las cláusulas indexadas $B[\mathbf{q}](\alpha_1, \dots, \alpha_k) \rightarrow \varepsilon$, una por cada asignación de rangos a sus argumentos. El argumento ε realiza solo el rango (q, q) para cada estado q , así que la cláusula $L_0(\varepsilon) \rightarrow \varepsilon$ produce ocho cláusulas en G' , una por cada estado del autómata:

$$\begin{aligned} L_0[(q_0, q_0)](\varepsilon) &\rightarrow \varepsilon, & L_0[(q_1, q_1)](\varepsilon) &\rightarrow \varepsilon, \\ L_0[(q_2, q_2)](\varepsilon) &\rightarrow \varepsilon, & L_0[(V, V)](\varepsilon) &\rightarrow \varepsilon, \end{aligned}$$

y de igual forma para los estados q_3, q_4, q_5 y F .

El conjunto de cláusulas P' consta de las cláusulas anteriores y de las del predicado inicial S' , que dependen de la condición de aceptación de A : una cadena se acepta cuando el rango que realiza parte del estado inicial q_0 y termina en un estado final. Por eso la construcción solo usa los predicados indexados $S[(q_0, q_F)]$ con $q_F \in F$, y, para cada uno, G' tiene una cláusula

$$S'(X) \rightarrow S[(q_0, q_F)](X),$$

donde S es el predicado inicial de G . Una cadena w pertenece a $L(G')$ si $S'(w)$ se deriva a la cadena vacía.

En el ejemplo de G_{cons} sobre A , $F = \{V\}$ ⁴, y el rango (q_0, V) parte del estado inicial y termina en el estado de aceptación. El predicado inicial S' tiene la cláusula:

$$S'(X) \rightarrow S[(q_0, V)](X).$$

La construcción conserva todas las cláusulas que produce, tanto las que provienen de las cláusulas de G como las del predicado inicial S' . Las que participan en el reconocimiento de una cadena se determinan durante la derivación, no durante la construcción.

La construcción produce los predicados N' , las cláusulas P' y el predicado inicial S' , y mantiene los terminales T y las variables V de G . Los cinco componentes completan la tupla $G' = (N', T, V, P', S')$, con lo que la construcción de la gramática producto termina.

Para ilustrar el reconocimiento en la gramática producto G' , se describe la derivación de la cadena $101 \in L(G_{\text{cons}}) \cap L(A)$, el testigo de que la fórmula es satisfacible. Cada predicado se indexa por el rango que su argumento realiza en la lectura de 101

⁴Aquí F es el conjunto de estados de aceptación del autómata, no el estado F del autómata booleano.

por A , cuyo camino es $q_0 1 q_1 0 q_2 1 V$:

$$\begin{aligned}
 S'(101) &\rightarrow S[(q_0, V)](101), \\
 S[(q_0, V)](101) &\rightarrow Eq_{x_1}[(q_0, V)](101) Eq_{x_2}[(q_0, V)](101), \\
 Eq_{x_1}[(q_0, V)](101) &\rightarrow L_0[(q_0, q_0)](\varepsilon) L_1[(q_1, q_2)](0) L_0[(V, V)](\varepsilon), \\
 Eq_{x_2}[(q_0, V)](101) &\rightarrow L_1[(q_0, q_1)](1) L_1[(q_2, V)](1), \\
 L_1[(q_1, q_2)](0) &\rightarrow L_0[(q_2, q_2)](\varepsilon), \\
 L_1[(q_0, q_1)](1) &\rightarrow L_0[(q_1, q_1)](\varepsilon), \\
 L_1[(q_2, V)](1) &\rightarrow L_0[(V, V)](\varepsilon),
 \end{aligned}$$

y cada predicado indexado $L_0[(q, q)](\varepsilon)$ deriva en la cadena vacía por la cláusula terminal. La figura 5.2 muestra el árbol de derivación, con cada predicado indexado por el rango que realiza su argumento. El primer paso aplica la cláusula del predicado inicial S' ; el segundo, la cláusula no lineal de S , que asocia el mismo rango (q_0, V) a las dos ocurrencias de X . Como la derivación de $S'(101)$ termina en la cadena vacía, G' reconoce 101.

La derivación de 101 usa solo unos pocos de los predicados y cláusulas de G' . La construcción genera muchos más que no aparecen en el reconocimiento de ninguna cadena de $L(G_{\text{cons}}) \cap L(A)$: los predicados indexados cuyo rango no parte de q_0 ni termina en V , como $Eq_{x_1}[(q_0, F)]$, quedan sin uso, junto con las cláusulas que los tienen en el lado izquierdo.

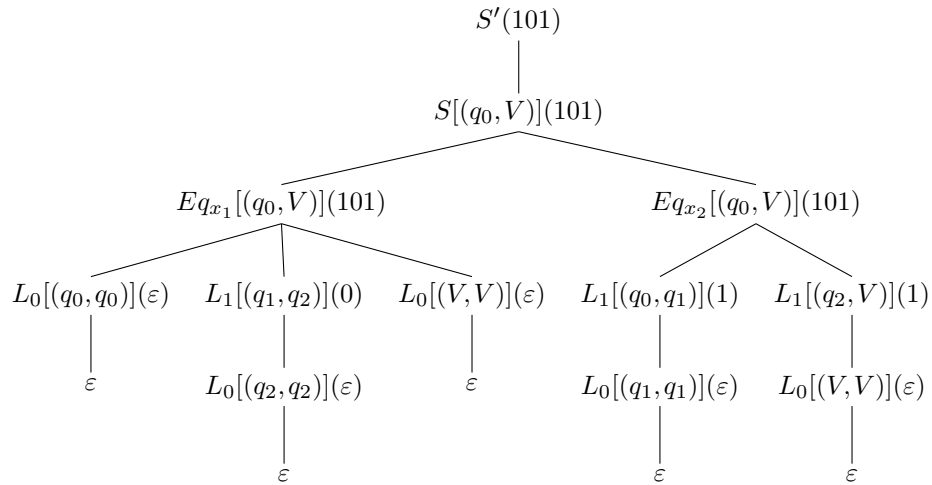


Figura 5.2: Árbol de derivación de 101 en la gramática producto G' . Cada predicado va indexado por el rango que realiza su argumento; las hojas ε corresponden a las cláusulas terminales $L_0[(q, q)](\varepsilon) \rightarrow \varepsilon$.

Por conveniencia, la construcción anterior se restringe a una clase de gramáticas

de entrada que evita casos aparte. La sección siguiente la precisa.

5.3.2. Restricción sobre la gramática de entrada

Cada predicado se indexa por los rangos sobre A de sus argumentos. El lado derecho de una cláusula fija los rangos de las variables que contiene. Si una variable apareciera en el lado izquierdo pero no en el lado derecho, el lado derecho no fijaría su rango, y habría que tratarla como un caso aparte.

Para evitarlo, las gramáticas de entrada se restringen a las que cumplen una propiedad sobre sus cláusulas: en cada cláusula, toda variable del lado izquierdo aparece también en el lado derecho.

Definición 5.5. Una gramática de concatenación de rango es **no borrante** si en cada cláusula toda variable del lado izquierdo aparece también en el lado derecho.

Se pide además que los argumentos del lado derecho sean variables, de modo que cada variable del lado derecho sea un argumento por sí sola.

Definición 5.6. Una gramática de concatenación de rango es **no combinatoria** si en cada cláusula los argumentos del lado derecho son variables.

Por ejemplo, las cláusulas de G_{cons} cumplen ambas: en $Eq_{x_1}(W_0 1 W_1 1 W_2) \rightarrow L_0(W_0) L_1(W_1) L_0(W_2)$ las variables W_0, W_1, W_2 del lado izquierdo aparecen en el lado derecho, cada una como un argumento por sí sola, y $L_0(\varepsilon) \rightarrow \varepsilon$ no tiene variables. Una cláusula como $A(XY) \rightarrow B(X)$ no cumple la condición de no borrante, pues Y figura en el lado izquierdo y falta en el lado derecho, y $A(X) \rightarrow B(X1)$ no cumple la de no combinatoria, pues el argumento del lado derecho $X1$ no es una variable.

Exigir ambas propiedades no descarta ningún lenguaje: toda gramática de concatenación de rango admite una equivalente que las cumple [6, 16].

Además de la restricción sobre la gramática de entrada, hace falta una condición sobre los rangos que se asignan, que se detalla a continuación.

5.3.3. Variables repetidas en una cláusula

En las gramáticas de concatenación de rango, una misma variable puede aparecer varias veces en una cláusula. En la construcción de la sección 5.3.1, cada argumento del lado derecho se indexa por separado, sin que se fije el mismo rango para las ocurrencias de una misma variable, que representan una sola cadena. Sin una condición que lo imponga, se les pueden asignar rangos distintos, y entonces G' reconocería cadenas que no están en la intersección.

Por eso es necesario añadir la siguiente condición: en cada cláusula, a todas las ocurrencias de una variable se les asigna el mismo rango. Más formalmente, si una

variable X aparece en dos argumentos de la cláusula con asignaciones de rangos ρ y ρ' , entonces $\rho(X) = \rho'(X)$.

Por ejemplo, la cláusula no lineal de G_{cons} ,

$$S(X) \rightarrow Eq_{x_1}(X) Eq_{x_2}(X),$$

tiene la variable X en tres argumentos: el del lado izquierdo y los dos del lado derecho. Sobre A , la cadena 000 es consistente, pero hace falsa la fórmula: realiza (q_0, F) , que no termina en un estado de aceptación.

Sin esta condición existe la cláusula

$$S[(q_0, V)](X) \rightarrow Eq_{x_1}[(q_0, F)](X) Eq_{x_2}[(q_0, F)](X),$$

con el predicado del lado izquierdo indexado en (q_0, V) y los del lado derecho en (q_0, F) . Como 000 realiza (q_0, F) y es consistente, los dos predicados Eq del lado derecho se derivan en la cadena vacía; con la cláusula $S'(X) \rightarrow S[(q_0, V)](X)$, la gramática G' reconocería 000, que está fuera de la intersección.

Con la condición, las tres ocurrencias de X llevan el mismo rango, así que la única cláusula cuyo lado izquierdo se indexa en (q_0, V) es

$$S[(q_0, V)](X) \rightarrow Eq_{x_1}[(q_0, V)](X) Eq_{x_2}[(q_0, V)](X).$$

Como 000 realiza (q_0, F) y no (q_0, V) , el predicado $Eq_{x_1}[(q_0, V)](000)$ no se deriva en la cadena vacía, así que 000 queda fuera del lenguaje de G' , como debería ser.

La repetición de variables es justo lo que separa a las RCG de las sRCG, y la condición no hace falta para estas últimas: en una sRCG cada variable aparece una sola vez en el lado derecho, así que no hay varias ocurrencias que coordinar. Imponerla es lo que permite tratar RCG y no solo sRCG.

Ya se presentaron la idea del algoritmo, la restricción sobre la gramática de entrada y la condición sobre las ocurrencias de una misma variable; con ellas se fija la construcción. A continuación se describe el pseudocódigo.

5.3.4. Pseudocódigo

El procedimiento PRODUCT-GRAMMAR recibe una gramática de concatenación de rango no combinatoria y no borrante $G = (N, T, V, P, S)$ y un autómata finito determinista acíclico $A = (Q, \Sigma, \delta, q_0, F)$; devuelve $G' = (N', T, V, P', S')$ con $L(G') = L(G) \cap L(A)$.

PRODUCT-GRAMMAR(G, A)

```

1   $P' = \emptyset$ 
2  for each clause  $c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow$ 
     $B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$  in  $P$ 
3      let  $\sigma_1, \sigma_2, \dots$  be the symbol occurrences of  $c$  and its empty arguments
4       $\ell =$  number of these
5      for each  $\mu = ((q_1, q'_1), \dots, (q_\ell, q'_\ell)) \in (Q \times Q)^\ell$ 
6          if CONSISTENT( $c, \mu$ )
7              for  $j = 0$  to  $m$ 
8                   $\mathbf{q}_j = (\text{RANGE}(\alpha_{j,1}, \mu), \dots, \text{RANGE}(\alpha_{j,k_j}, \mu))$ 
9                  add  $B_0[\mathbf{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow$ 
                       $B_1[\mathbf{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\mathbf{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m})$  to  $P'$ 
10 for each  $q \in F$ 
11     add  $S'(X) \rightarrow S[(q_0, q)](X)$  to  $P'$ 
12  $N' = \{S'\} \cup \{B' : B' \text{ occurs in } P'\}$ 
13 return ( $N', T, V, P', S'$ )

```

En la línea 2 se recorren las cláusulas de G . Para cada una, en las líneas 3 y 4 se cuentan los símbolos que aparecen en ella, con repeticiones, y sus argumentos vacíos. En la línea 5 se recorren todas las posibles asignaciones de rangos. De la línea 6 a la 9, con CONSISTENT se conservan solo las asignaciones en las que cada rango termina donde empieza el siguiente, cada terminal a con rango (q, q') cumple $\delta(q, a) = q'$, y todas las ocurrencias de una variable reciben el mismo rango; por cada una se indexa cada predicado por los rangos de sus argumentos con RANGE, y se agrega la cláusula a P' . En las líneas 10 y 11 se agregan las cláusulas del predicado inicial S' , una por cada estado final. En la línea 12 se agregan a N' el predicado S' y los predicados que aparecen en P' .

En CONSISTENT se comprueban las condiciones sobre μ :

CONSISTENT(c, μ)

```

1  for each non-empty argument  $\alpha = \alpha_1 \cdots \alpha_n$  of  $c$ , with  $\mu(\alpha_i) = (q_i, q'_i)$ 
2      for  $i = 1$  to  $n$ 
3          if  $i < n$  and  $q'_i \neq q_{i+1}$ 
4              return FALSE
5          if  $\alpha_i$  is a terminal  $a$  and  $\delta(q_i, a) \neq q'_i$ 
6              return FALSE
7  for each empty argument  $\alpha$  of  $c$ , with  $\mu(\alpha) = (q, q')$ 
8      if  $q \neq q'$ 
9          return FALSE
10 for each variable  $X$  of  $c$ 
11     if two occurrences of  $X$  have different ranges under  $\mu$ 
12         return FALSE
13 return TRUE

```

En las líneas 1–9 de CONSISTENT se comprueban las condiciones sobre los rangos de los argumentos; en las líneas 10–12, que las ocurrencias de una variable reciban el mismo rango.

RANGE(α, μ) es el rango que el argumento α realiza bajo μ :

RANGE(α, μ)

```

1  if  $\alpha = \varepsilon$ 
2      return  $\mu(\alpha)$ 
3  let  $\alpha = \alpha_1 \cdots \alpha_n$ , with  $\mu(\alpha_i) = (q_i, q'_i)$ 
4  return  $(q_1, q'_n)$ 

```

Para $\alpha = \alpha_1 \cdots \alpha_n$ es (q_1, q'_n) , el inicio del primer símbolo y el final del último; para $\alpha = \varepsilon$ es el rango (q, q) que μ le asigna, el rango que α realiza según la sección 5.2.

En la próxima subsección se demuestra que G' reconoce exactamente $L(G) \cap L(A)$.

5.3.5. Correctitud

En toda la sección G es no combinatoria y no borrante. La igualdad $L(G') = L(G) \cap L(A)$ se prueba por las dos inclusiones. La clave es una propiedad de los índices: una cláusula indexada solo reconoce cadenas que realizan los rangos con que se indexan sus argumentos. De ahí, A acepta toda cadena que G' reconoce; y, al borrar los índices, también la reconoce G . En el otro sentido, se parte de una derivación de G y se indexa cada predicado con los rangos sobre A que sus argumentos realizan al leer la cadena; el resultado es una derivación de G' .

Sea $B_0[\mathbf{q}_0](u_1, \dots, u_{k_0})$ el predicado indexado $B_0[\mathbf{q}_0]$ aplicado a las cadenas $u_1, \dots, u_{k_0}$, una por cada argumento, con $\mathbf{q}_0 = ((q_1, q'_1), \dots, (q_{k_0}, q'_{k_0}))$.

Lema 5.1. *Si $B_0[\mathbf{q}_0](u_1, \dots, u_{k_0})$ se deriva a ε en G' , entonces $B_0(u_1, \dots, u_{k_0})$ se deriva a ε en G .*

Demostración. Las cláusulas que intervienen en una derivación de $B_0[\mathbf{q}_0](u_1, \dots, u_{k_0})$ son cláusulas indexadas, cada una proveniente de una cláusula de P a la que solo se le agregan los índices de los predicados; el predicado S' no aparece, pues no figura en el lado derecho de ninguna cláusula. Al borrar los índices, cada cláusula indexada se reduce a su cláusula de P , y se obtiene una derivación de $B_0(u_1, \dots, u_{k_0})$ en G . $\square$

La hipótesis de que G es no combinatoria es clave en el segundo lema.

Lema 5.2. *Si $B_0[\mathbf{q}_0](u_1, \dots, u_{k_0})$ se deriva a ε en G' , entonces cada u_l realiza el rango (q_l, q'_l) .*

Demostración. La prueba es por inducción en la cantidad de pasos de la derivación de $B_0[\mathbf{q}_0](u_1, \dots, u_{k_0})$ a ε . La hipótesis de inducción es que el lema vale para toda derivación con menos pasos.

La derivación comienza con una cláusula de P' de lado izquierdo $B_0[\mathbf{q}_0]$, que proviene de una cláusula de P ,

$$c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m}),$$

y de una asignación μ de un rango a cada ocurrencia de símbolo de c . Recuerdese que esta asignación cumple tres condiciones. La primera es que en cada argumento el final del rango de un símbolo sea el inicio del siguiente. La segunda es que cada terminal a con rango (q, q') cumpla $\delta(q, a) = q'$. La tercera es que todas las ocurrencias de una variable reciban el mismo rango. La cláusula indexada que resulta es

$$B_0[\mathbf{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1[\mathbf{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\mathbf{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m}),$$

con (q_l, q'_l) el rango que el argumento $\alpha_{0,l}$ del lado izquierdo realiza bajo μ . La cláusula se instancia con una sustitución θ de las variables por subcadenas, de modo que $u_l = \theta(\alpha_{0,l})$.

En el caso base, el lado derecho de c es vacío ($m = 0$), de modo que

$$c = B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow \varepsilon,$$

y la derivación tiene un solo paso. Como el lado derecho es vacío y G es no borrante, el lado izquierdo no tiene variables, pues toda variable suya tendría que aparecer en el lado derecho. Así, cada argumento $\alpha_{0,l}$ es una cadena de terminales, y por la primera y la segunda condición $u_l = \alpha_{0,l}$ realiza (q_l, q'_l) .

En el paso inductivo, el lado derecho no es vacío, y cada predicado

$$B_j[\mathbf{q}_j](\theta(\alpha_{j,1}), \dots, \theta(\alpha_{j,k_j}))$$

se deriva a ε en menos pasos. Como G es no combinatoria, cada argumento del lado derecho es una sola variable X , con un único rango $\mu(X)$ en todas sus ocurrencias por la tercera condición, y el rango que ese argumento realiza es $\mu(X)$. Por la hipótesis de inducción, $\theta(X)$ realiza $\mu(X)$ para toda variable X del lado derecho; como G es no borrante, lo mismo vale para toda variable de c .

Sea entonces $\alpha_{0,l}$ un argumento del lado izquierdo. Por la segunda condición, cada terminal de $\alpha_{0,l}$ realiza el rango que μ le asigna; y para cada variable X , $\theta(X)$ realiza $\mu(X)$, que por la tercera condición es el rango de todas sus ocurrencias. Como el final de cada rango es el inicio del siguiente, por la primera condición, la concatenación de estas subcadenas, $u_l = \theta(\alpha_{0,l})$, realiza la concatenación de sus rangos, (q_l, q'_l) . Si $\alpha_{0,l} = \varepsilon$, su rango (q_l, q'_l) tiene $q_l = q'_l$, por ser el de un argumento vacío, y $u_l = \varepsilon$ lo realiza. $\square$

Con los dos lemas se obtiene el teorema.

Teorema 5.1. $L(G') = L(G) \cap L(A)$.

Demostración. Inclusión $L(G') \subseteq L(G) \cap L(A)$. Sea $w \in L(G')$; entonces $S'(w)$ se deriva a ε . Las únicas cláusulas de lado izquierdo S' son $S'(X) \rightarrow S[(q_0, q_F)](X)$ con $q_F \in F$, así que el primer paso es $S'(w) \rightarrow S[(q_0, q_F)](w)$ para algún $q_F \in F$, y $S[(q_0, q_F)](w)$ se deriva a ε . Por el lema 5.1, $S(w)$ se deriva a ε en G , esto es, $w \in L(G)$. Por el lema 5.2, w realiza (q_0, q_F) , es decir, $\hat{\delta}(q_0, w) = q_F$; como $q_F \in F$, $w \in L(A)$. Luego $w \in L(G) \cap L(A)$.

Inclusión $L(G) \cap L(A) \subseteq L(G')$. Sea $w = a_1 \cdots a_n \in L(G) \cap L(A)$. El camino de la lectura de w por A es $p_0 a_1 p_1 \cdots a_n p_n$, con $p_0 = q_0$ y $p_i = \delta(p_{i-1}, a_i)$; como $w \in L(A)$, $p_n \in F$. La subcadena en las posiciones i a j realiza el rango sobre A (p_i, p_j) .

Como $w \in L(G)$, hay una derivación de $S(w)$ a ε en G , en la que cada predicado se aplica a subcadenas de w en posiciones fijas. Al indexar cada predicado por los rangos sobre A de sus argumentos se obtiene una derivación en G' , porque cada cláusula indexada pertenece a P' . En efecto, sea

$$B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$$

una cláusula instanciada en la derivación, y μ la asignación que a cada ocurrencia de símbolo le asigna el rango sobre A de la posición que ocupa. La asignación μ cumple las tres condiciones: los símbolos contiguos de un argumento ocupan posiciones contiguas, así que el final de cada rango es el inicio del siguiente, y un argumento vacío en la posición i ocupa el rango (p_i, p_i) ; para un terminal a en la posición $i+1$ se tiene $\delta(p_i, a) = p_{i+1}$; y las ocurrencias de una variable ocupan las mismas posiciones, luego reciben el mismo rango. Por tanto, la construcción produce la cláusula indexada, que pertenece a P' .

El lado izquierdo $S(w)$ se indexa por el rango $(p_0, p_n) = (q_0, p_n)$. Como $p_n \in F$, la cláusula $S'(X) \rightarrow S[(q_0, p_n)](X)$ pertenece a P' , así que $S'(w)$ se deriva a ε y $w \in L(G')$. $\square$

La gramática producto es correcta, pero su tamaño puede ser exponencial en la entrada, como se ve a continuación.

5.3.6. Tamaño y complejidad

El tamaño de G' y el costo de construirla dependen del número de predicados $|N|$ y cláusulas $|P|$ de G , de los $|Q|$ estados de A , de la aridad máxima a y del máximo ℓ de ocurrencias de símbolo y argumentos vacíos por cláusula.

Cada predicado de aridad k se indexa con un vector de k rangos sobre A . Un rango es un par de estados, de modo que hay $|Q|^2$ rangos y $(|Q|^2)^k = |Q|^{2k}$ vectores posibles. Como la aridad máxima es a , por cada uno de los $|N|$ predicados hay a lo sumo $|Q|^{2a}$ predicados indexados. Con el predicado inicial,

$$|N'| = O(|N||Q|^{2a}).$$

Para cada cláusula con ℓ_c ocurrencias de símbolo se recorren, en la línea 5 del procedimiento PRODUCT-GRAMMAR, las $|Q|^{2\ell_c}$ asignaciones de un rango a cada una, y por cada una que cumple las tres condiciones se agrega a lo sumo una cláusula indexada; las del predicado inicial son una por estado final. Así,

$$|P'| = O(|P||Q|^{2\ell} + |F|) = O(|P||Q|^{2\ell}).$$

Como los terminales son los de G , el tamaño de G' es

$$|G'| = |N'| + |T| + |P'| = O(|N||Q|^{2a} + |P||Q|^{2\ell} + |T|).$$

El costo de construir G' es el número de asignaciones recorridas por el $O(\ell_c)$ de verificar y procesar cada una:

$$O(|P|\ell|Q|^{2\ell}).$$

Los factores $|Q|^{2a}$ y $|Q|^{2\ell}$ son exponenciales en la aridad máxima a y en ℓ . Por eso el tamaño de G' es exponencial cuando a o ℓ no están acotados, y el costo de construirla, con solo el factor $|Q|^{2\ell}$, lo es cuando ℓ no está acotado. El valor de ℓ crece con la cantidad de variables de un mismo argumento del lado izquierdo, pues por cada una queda libre el rango con que se la indexa.

En la modelación del capítulo 4, G_{cons} es la gramática de consistencia de una fórmula con n ocurrencias de variable booleana y A su autómata booleano, con

$|N| = O(n)$, $|P| = O(n)$, $|Q| = O(n^2)$ y $a = 1$. El número de predicados indexados es polinomial,

$$|N'| = O(n(n^2)^2) = O(n^5).$$

El número de cláusulas no lo es: la cláusula de Eq_v para una variable booleana con r ocurrencias,

$$Eq_v(W_0 b W_1 b \cdots b W_r) \rightarrow L_{g_0}(W_0) L_{g_1}(W_1) \cdots L_{g_r}(W_r),$$

tiene $r + 1$ variables en su argumento del lado izquierdo, y r puede ser $O(n)$, de modo que $\ell = O(n)$. Entonces

$$|P'| = O\left(n(n^2)^{O(n)}\right) = 2^{O(n \log n)}.$$

El tamaño de la gramática producto de G_{cons} y el autómata booleano es

$$|G'| = |N'| + |T| + |P'| = O(n^5) + O(1) + 2^{O(n \log n)} = 2^{O(n \log n)}.$$

Construirla cuesta lo mismo:

$$O(|P| \ell |Q|^{2\ell}) = O\left(n^2 (n^2)^{O(n)}\right) = 2^{O(n \log n)}.$$

Las cláusulas de Eq_v son el término dominante.

En SATSRC el grado del polinomio con que se decide el vacío de la intersección crece con la aridad de la sRCG, como se ve en la subsección 2.3.2 del capítulo 2. La aridad $a = 1$ de G_{cons} se debe a su no linealidad: un mismo argumento aparece en varios predicados del lado derecho, y así la consistencia de cada variable booleana se reconoce sin aumentar la aridad. A cambio se pierde la tratabilidad del vacío de la intersección, polinomial con una sRCG y NP-completo con una RCG no lineal.

Construir G' sirve para decidir el vacío de la intersección, el problema del capítulo 3. Como $L(G') = L(G) \cap L(A)$, esa intersección es vacía si y solo si lo es $L(G')$, esto es, si el predicado inicial S' no es productivo. Para esa decisión solo importan los predicados y cláusulas productivos, y muchos de los indexados no lo son: no intervienen en el reconocimiento de ninguna cadena de $L(G_{\text{cons}}) \cap L(A)$, como se observó en la sección 5.3.1. En este trabajo no se da el costo de decidir ese vacío, sino solo el objeto G' ; limpiarlo de los predicados y cláusulas no productivos y analizar ese costo quedan como problemas abiertos.

En este capítulo se construyó la gramática producto G' de una RCG G y un autómata finito acíclico A , se demostró que $L(G') = L(G) \cap L(A)$, se caracterizaron su tamaño y el costo de construirla, exponenciales en el peor caso, y se mostró que muchos de los predicados y cláusulas de G' no son productivos. En el próximo capítulo se presentan las conclusiones y recomendaciones del trabajo.

Conclusiones y recomendaciones

En este trabajo se estudió el vacío de la intersección de una gramática de concatenación de rango con un lenguaje regular finito como marco para el problema de la satisfacibilidad booleana.

En el capítulo 2 se presentaron las gramáticas de concatenación de rango y los antecedentes en que el SAT se aborda como la intersección de un lenguaje regular de interpretaciones con una gramática que impone la consistencia entre las ocurrencias de cada variable, en SATCF y SATSRC.

En el capítulo 3 se definió el problema del vacío de la intersección de una RCG con un lenguaje regular finito y se dio un algoritmo de decisión, en el que se enumeran las cadenas del autómata y se reconoce cada una con la gramática. Con él se decide el problema, pero su costo es exponencial cuando la cantidad de cadenas aceptadas por el autómata es exponencial, y polinomial cuando esa cantidad está acotada por un polinomio.

En el capítulo 4 ese resultado se aplicó al SAT: a cada fórmula booleana se le asocian un autómata booleano y una gramática de consistencia, de modo que la fórmula es satisfacible si y solo si la intersección de sus lenguajes no es vacía. Decidir ese vacío es el SAT, y para una RCG con un autómata finito acíclico ese problema es NP-completo [4].

En el capítulo 5 se construyó la gramática producto G' de una RCG G y un autómata finito acíclico A , con $L(G') = L(G) \cap L(A)$, y se demostró su correctitud. La gramática producto es una RCG sobre la que estudiar el vacío de la intersección, pero su tamaño puede ser exponencial: para la gramática de consistencia y el autómata booleano de una fórmula con n ocurrencias de variable booleana, $|G'| = 2^{O(n \log n)}$, y las cláusulas de la consistencia son el término dominante.

En conjunto, estos resultados son un marco uniforme para el SAT en torno a una sola noción: el vacío de la intersección de una RCG con un lenguaje regular finito.

La gramática de consistencia del capítulo 4 tiene aridad uno, para cualquier fórmula, gracias a la no linealidad. Para abarcar más fórmulas que SATSRC sin que la aridad crezca con los cruces entre ocurrencias hace falta la no linealidad, pues sin ella se reconocen los mismos lenguajes que con una sRCG, como se vio en el capítulo 2. Por esa misma no linealidad la modelación queda fuera de las subclases conocidas con

vacío de la intersección polinomial, como SATCF y SATSRC, y se pierde la garantía directa de que el vacío sea polinomial. Si solo se modelara una subclase de fórmulas, el vacío podría ser polinomial sin que ello implique $P = NP$; decidirlo en tiempo polinomial para todas las fórmulas sí implicaría $P = NP$.

La gramática producto del capítulo 5 es el objeto sobre el que abordar ese vacío: hallar un método polinomial para todas las fórmulas, probar que no existe, o hallarlo solo al restringir la modelación a una subclase. Para decidir el vacío en tiempo polinomial sobre la gramática producto hace falta eliminar antes sus predicados y cláusulas no productivos.

A partir del trabajo realizado se proponen las siguientes líneas de investigación:

- Eliminar de la gramática producto los predicados y cláusulas no productivos y analizar la complejidad de decidir el vacío de la intersección sobre la gramática resultante.
- Buscar subclases no lineales de RCG cuyo vacío de la intersección con un lenguaje regular finito se decida en tiempo polinomial, y estudiar si la gramática de consistencia se adapta a alguna de ellas.
- Modificar las RCG para obtener un formalismo cuyo vacío de la intersección con un lenguaje regular finito se decida en tiempo polinomial y con el que se exprese la consistencia de fórmulas distintas de las que ya se modelan con SATCF y SATSRC.
- Caracterizar las fórmulas booleanas cuyo autómata booleano tiene una cantidad de caminos aceptantes acotada por un polinomio, donde el SAT se decide en tiempo polinomial con la enumeración del capítulo 3.

Referencias

- [1] Manuel Aguilera López. «Problema de la Satisfacibilidad Booleana de Concatenación de Rango Simple». En: *Jornada Científica Estudiantil MatCom 2016*. Facultad de Matemática y Computación, Universidad de La Habana. La Habana, Cuba, 2016, págs. 1-5 (vid. págs. 1, 18, 21, 25, 27).
- [2] Jossué Arteche Muñoz. «El problema de la satisfacibilidad booleana y el vacío de la intersección con lenguajes regulares». Trabajo de Diploma. Universidad de La Habana, Facultad de Matemática y Computación, mayo de 2026 (vid. pág. 25).
- [3] Yehoshua Bar-Hillel, Micha Perles y Eli Shamir. «On Formal Properties of Simple Phrase Structure Grammars». En: *Zeitschrift für Phonetik, Sprachwissenschaft und Kommunikationsforschung* 14.2 (1961), págs. 143-172 (vid. pág. 44).
- [4] Eberhard Bertsch y Mark-Jan Nederhof. «On the Complexity of Some Extensions of RCG Parsing». En: *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*. Beijing, China, 2001, págs. 66-77 (vid. págs. 2, 20, 27, 29, 42, 44, 59).
- [5] Pierre Boullier. «Chinese Numbers, MIX, Scrambling, and Range Concatenation Grammars». En: *Proceedings of the 9th Conference of the European Chapter of the Association for Computational Linguistics (EACL '99)*. Bergen, Norway, 1999, págs. 53-60 (vid. pág. 13).
- [6] Pierre Boullier. *Proposal for a Natural Language Processing Syntactic Backbone*. Inf. téc. RR-3342. France: INRIA-Rocquencourt, 1998, pág. 41 (vid. págs. 2, 13, 19, 20, 30, 33, 34, 51).
- [7] Pierre Boullier. «Range Concatenation Grammars». En: *Proceedings of the Sixth International Workshop on Parsing Technologies (IWPT 2000)*. Trento, Italy: Association for Computational Linguistics, 2000, págs. 53-64. URL: <https://aclanthology.org/2000.iwpt-1.8/> (vid. pág. 20).

- [8] Stephen A. Cook. «The complexity of theorem-proving procedures». En: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. New York, NY, USA: ACM, 1971, págs. 151-158. DOI: 10.1145/800157.805047 (vid. págs. 1, 12).
- [9] William F. Dowling y Jean H. Gallier. «Linear-Time Algorithms for Testing the Satisfiability of Propositional Horn Formulae». En: *Journal of Logic Programming* 1.3 (1984), págs. 267-284. DOI: 10.1016/0743-1066(84)90014-1 (vid. pág. 12).
- [10] Alina Fernández Arias. «El problema de la satisfacibilidad booleana libre del contexto. Un algoritmo polinomial». Tesis de Licenciatura. La Habana, Cuba: Facultad de Matemática y Computación, Universidad de La Habana, 2007 (vid. págs. 1, 7, 21, 23, 24, 27, 40, 41).
- [11] Raudel Alejandro Gómez Molina. «Una aproximación al lenguaje de todas las fórmulas booleanas satisfacibles mediante gramáticas de concatenación de rango». Tesis de Licenciatura. La Habana, Cuba: Facultad de Matemática y Computación, Universidad de La Habana, feb. de 2025 (vid. pág. 6).
- [12] Jun Gu et al. «Algorithms for the Satisfiability (SAT) Problem: A Survey». En: *Satisfiability Problem: Theory and Applications*. Ed. por Dingzhu Du, Jun Gu y Panos M. Pardalos. Vol. 35. DIMACS Series in Discrete Mathematics and Theoretical Computer Science. American Mathematical Society, 1997, págs. 19-151 (vid. pág. 1).
- [13] Steven Halim, Felix Halim y Suhendry Effendy. *Competitive Programming 4 – Book 2: The Lower Bound of Programming Contests in the 2020s*. 4.^a ed. Lulu.com, 2020. ISBN: 978-1-7167-4551-5 (vid. pág. 12).
- [14] John E. Hopcroft, Rajeev Motwani y Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3.^a ed. Boston, MA: Pearson Addison-Wesley, 2006, pág. 535. ISBN: 978-0-321-45536-9 (vid. págs. 5-12, 20, 44, 45).
- [15] Tim Hunter. «The Chomsky Hierarchy». En: *A Companion to Chomsky*. Ed. por Nicholas Allott, Terje Lohndal y Georges Rey. Wiley-Blackwell, 2021. Cap. 5, págs. 74-95. DOI: 10.1002/9781119598732.ch5 (vid. pág. 6).
- [16] Laura Kallmeyer. *Parsing Beyond Context-Free Grammars*. Cognitive Technologies. Berlin, Heidelberg: Springer, 2010. ISBN: 978-3-642-14845-3. DOI: 10.1007/978-3-642-14846-0 (vid. págs. 30, 33, 34, 51).
- [17] Thomas J. Schaefer. «The complexity of satisfiability problems». En: *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*. New York, NY, USA: ACM, 1978, págs. 216-226. DOI: 10.1145/800133.804350 (vid. págs. 1, 12).

- [18] K. Vijay-Shanker, David J. Weir y Aravind K. Joshi. «Characterizing Structural Descriptions Produced by Various Grammatical Formalisms». En: *25th Annual Meeting of the Association for Computational Linguistics*. Stanford, California, USA: Association for Computational Linguistics, 1987, págs. 104-111. DOI: 10.3115/981175.981190 (vid. pág. 18).